

The Kernel Abstraction

Challenge: Protection

- How do we execute code with restricted privileges?
 - Either because the code is buggy or if it might be malicious
- Some examples:
 - A script running in a web browser
 - A program you just downloaded off the Internet
 - A program you just wrote that you haven't tested yet

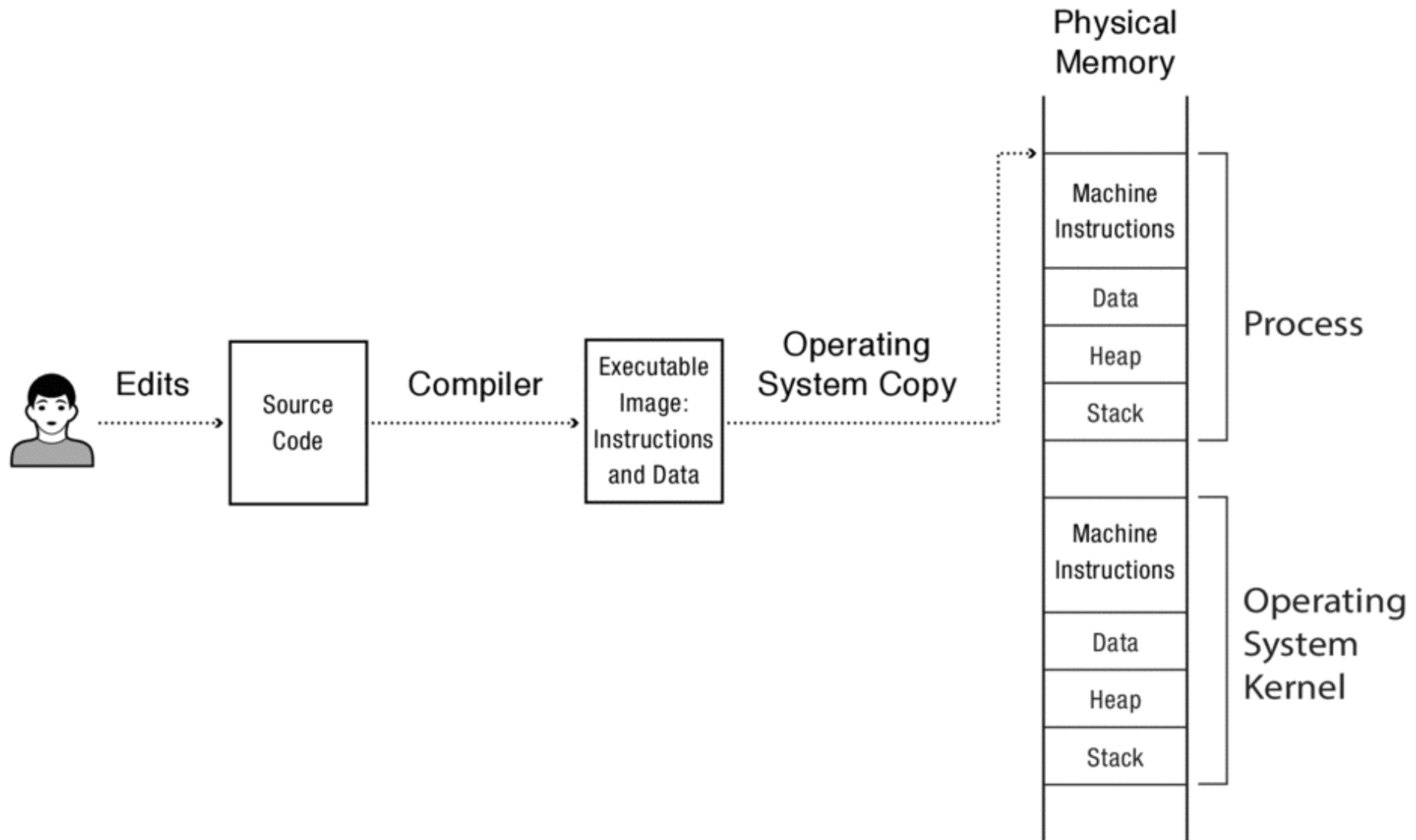
Thought Experiment

- Implementing execution with limited privileges:
 - Execute each program instruction in a simulator
 - If the instruction is permitted, do the instruction
 - Otherwise, stop the process
 - Basic model in Javascript, ...
- How do we go faster?
 - Run the unprivileged code directly on the CPU?

Main Points

- Process concept
 - A process is an OS abstraction for executing a program with limited privileges
- Dual-mode operation: *user vs. kernel mode*
 - *Kernel-mode*: execute with complete privileges
 - *User-mode*: execute with fewer privileges
- Safe control transfer
 - How do we switch from one mode to the other?

Process Concept



Process Concept

- Process: a sequence of activities activated by a program, **running with limited rights**
 - Process control block (PCB): the data structure the OS uses to keep track of a process
 - Process Table: contains all PCBs
 - Two elements:
 - Thread: executes a sequence of instructions within a process
 - Potentially many threads per process (for now 1:1)
 - Thread aka lightweight process
 - Address space: set of rights of a process
 - Memory that the process can access
 - Other permissions the process has (e.g., which procedure calls it can make, what files it can access)

Program and Process

- Program: a static sequence of instructions
- Process:
 - a dynamic entity: *a sequence of activities described by a program*
 - executed on a set of CPUs with limited rights
- Several processes can be activated on the same program
 - The processes execute the same code
 - Each process executes the program on different data and/or in different times

Process Control Block (PCB)

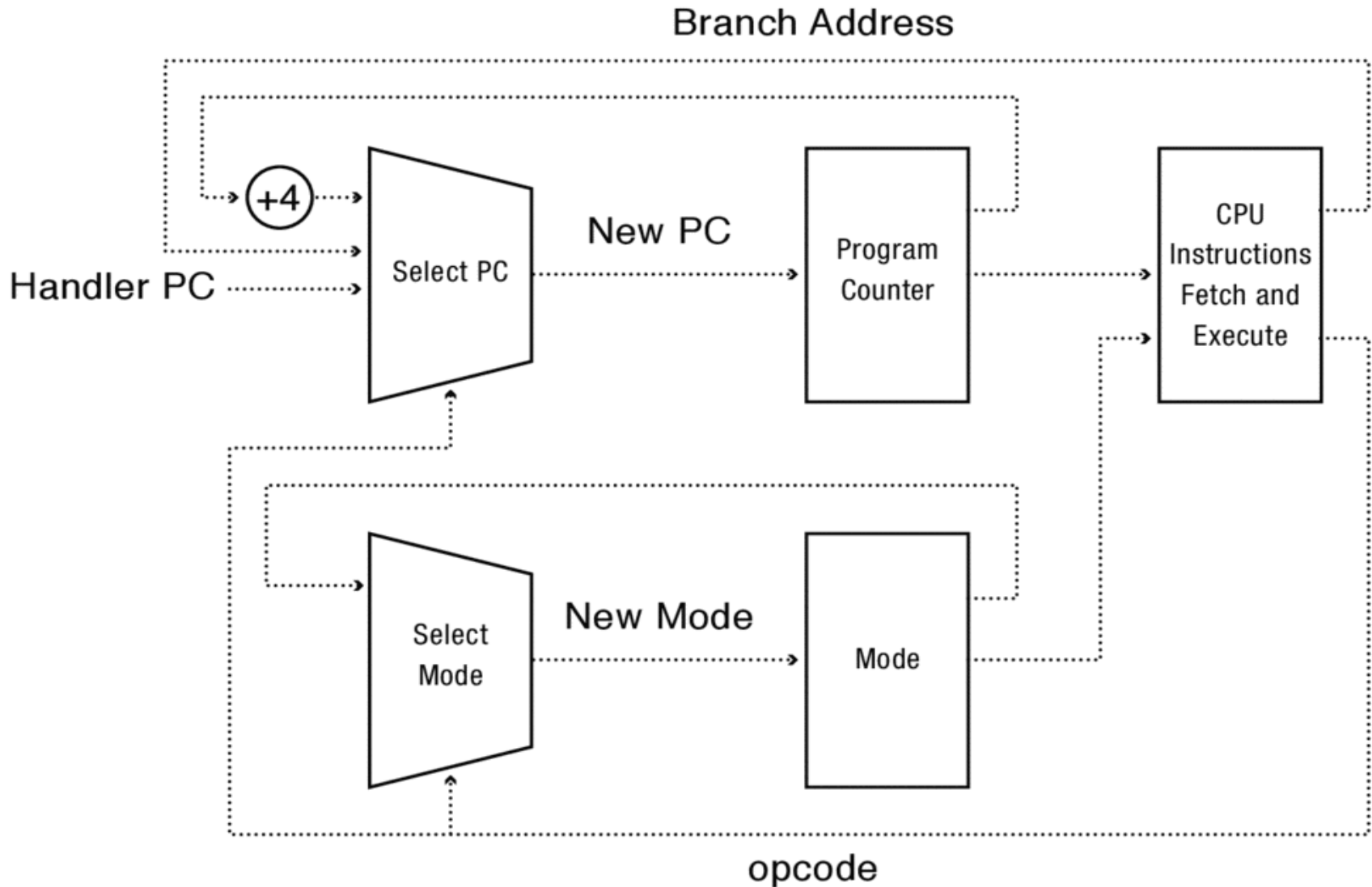
It is a data structure:

- Process name
 - Can be the index of the PCB in the process table
- Process state (new, ready, running, stopped,)
- CPU registers and scheduling information
- Pointers to process threads
- Assigned memory (memory limits)
- Other assigned resources
 - Open Files, devices, I/O status, etc...

Hardware Support: Dual-Mode Operation

- Kernel mode
 - Execution with the full privileges of the hardware
 - Read/write to any memory, access any I/O device, read/write any disk sector, send/read any packet
- User mode
 - Limited privileges
 - Only those granted by the operating system kernel
- On the x86, mode stored in EFLAGS register
- On the ARM, mode stored in CPSR register
 - In general, in the ***Program Status Register*** (PSR/PSW)

A CPU with Dual-Mode operation



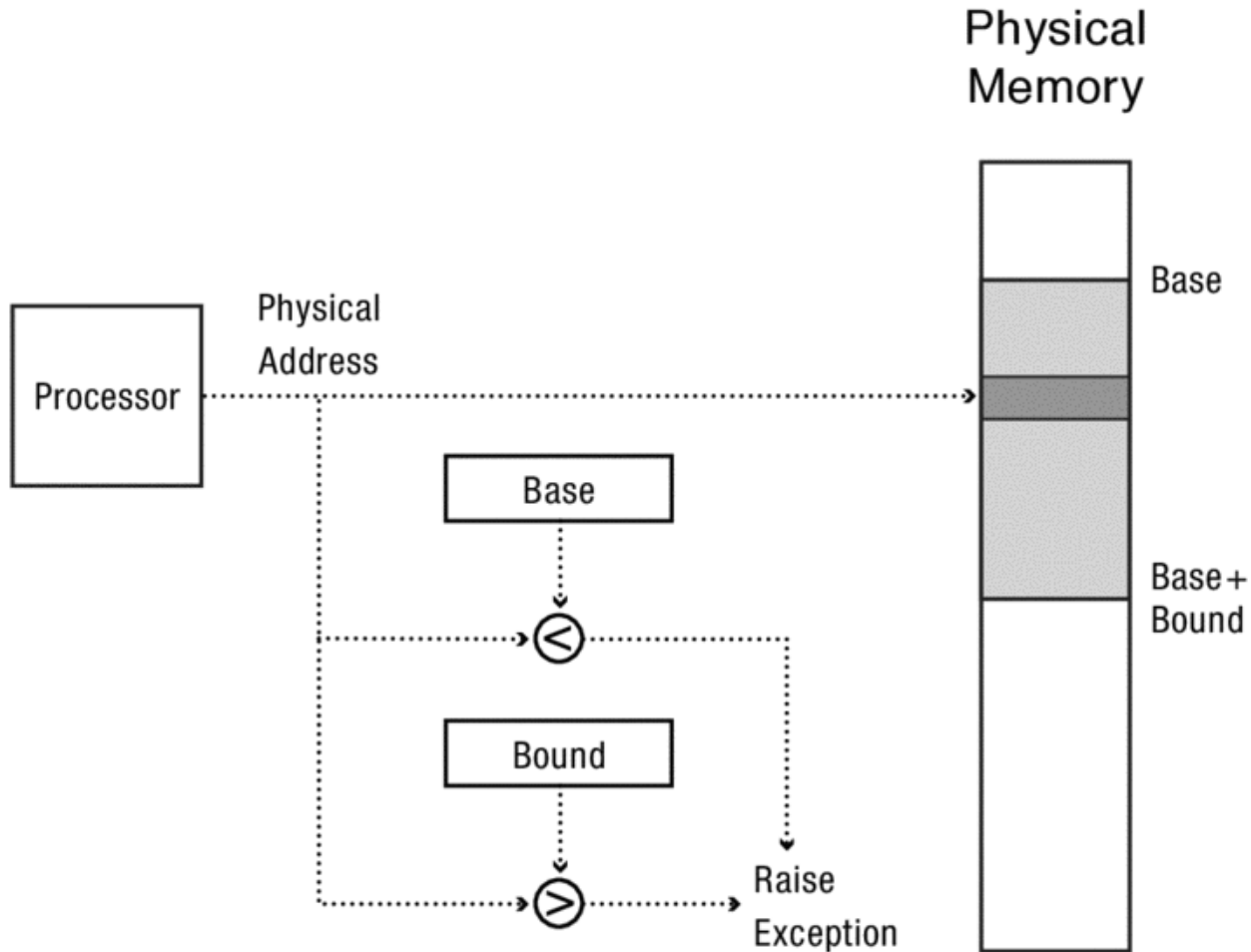
Hardware Support: Dual-Mode Operation

- Privileged instructions
 - Available to kernel
 - Not available to user code
- Limits on memory accesses (memory protection)
 - To prevent user code from overwriting the kernel
- Timer
 - To regain control from a user program in a loop
- Safe way to switch from user mode to kernel mode, and vice versa

Privileged instructions

- Examples?
 - Disabling interrupts
 - Setting the timer
 - Changing the bits in the PSW (CPSR)
 - ...
- What should happen if a user program attempts to execute a privileged instruction?
 - The OS takes over

Base-Bound Memory Protection with Physical addresses

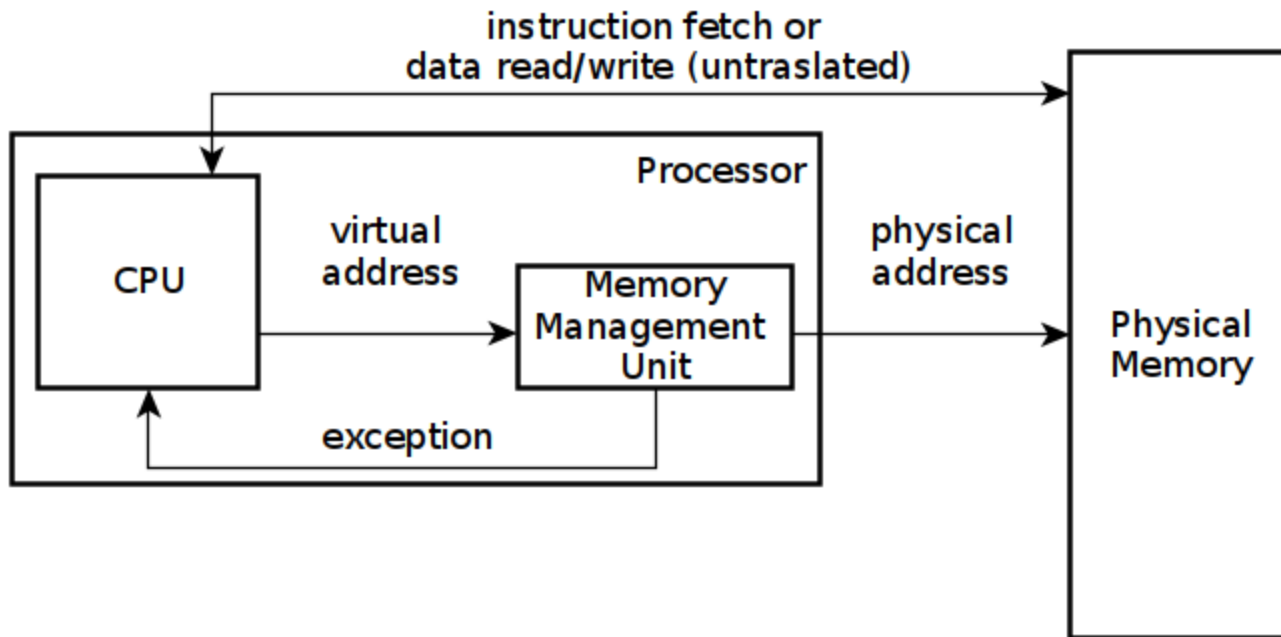


Towards Virtual Addresses

- Problems with base and bounds?
- No or limited expandable heap and stack
- No memory sharing
- Need to work with physical addresses -> hard to move memory around
- Memory fragmentation

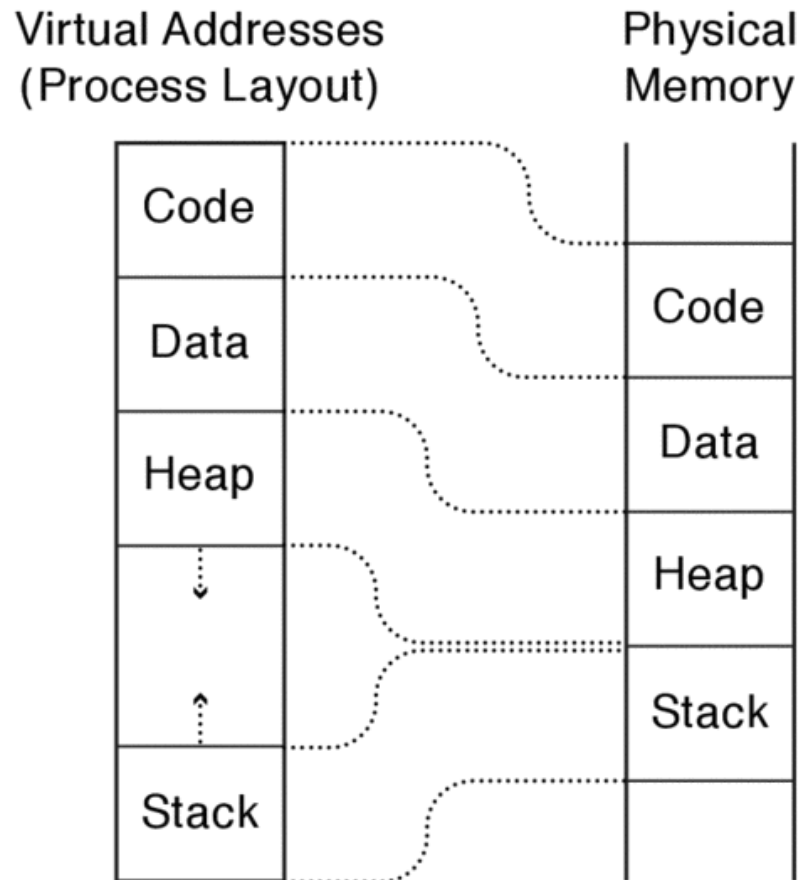
Virtual Addresses

- Translation done in hardware, using a table
- Table set up by the kernel
- Usually, the MMU takes care of address translation



Virtual Address Layout

- Plus shared code segments, dynamically linked libraries, memory mapped files, ...



Example

- What if we run two instances of this program at the same time?

```
int staticVar = 0;    // a static variable
main() {
    int localVar = 0; // a procedure local variable
    staticVar += 1; localVar += 1;
    sleep(10); // sleep causes the program to wait for 10 seconds
    printf ("static address: %p, value: %d\n", &staticVar, staticVar);
    printf ("local address: %p, value: %d\n", &localVar, localVar);
}
```

Produces (*):

static address: 0x100407000, value: 1

local address: 0xffffcc3c, value: 1

(*) Because of the *Address Space Layout Randomization* (ASLR) the output can be different at each run.....

Question

- Suppose we had a perfect object-oriented language and compiler, so that only an object's methods could access the internal data inside an object. If the operating system only ran programs written in that language, would it still need hardware memory address protection?

Hardware Timer

- Hardware device that periodically interrupts the processor
 - Returns control to the *kernel timer interrupt handler*
 - Interrupt frequency set by the kernel
 - Not by user code!
 - Interrupts can be temporarily deferred
 - Not by user code!
 - Interrupt deferral crucial for implementing mutual exclusion (we will see it later on)

Mode Switch

- From user-mode to kernel
 - Interrupts
 - Triggered by timer and I/O devices
 - Exceptions
 - Triggered by unexpected program behavior
 - Or malicious behavior!
 - System calls (aka protected procedure call)
 - Request by program for kernel to do some operation on its behalf
 - Only limited # of very carefully coded entry points
 - Also called Software Interrupts (SWIs)

Mode Switch

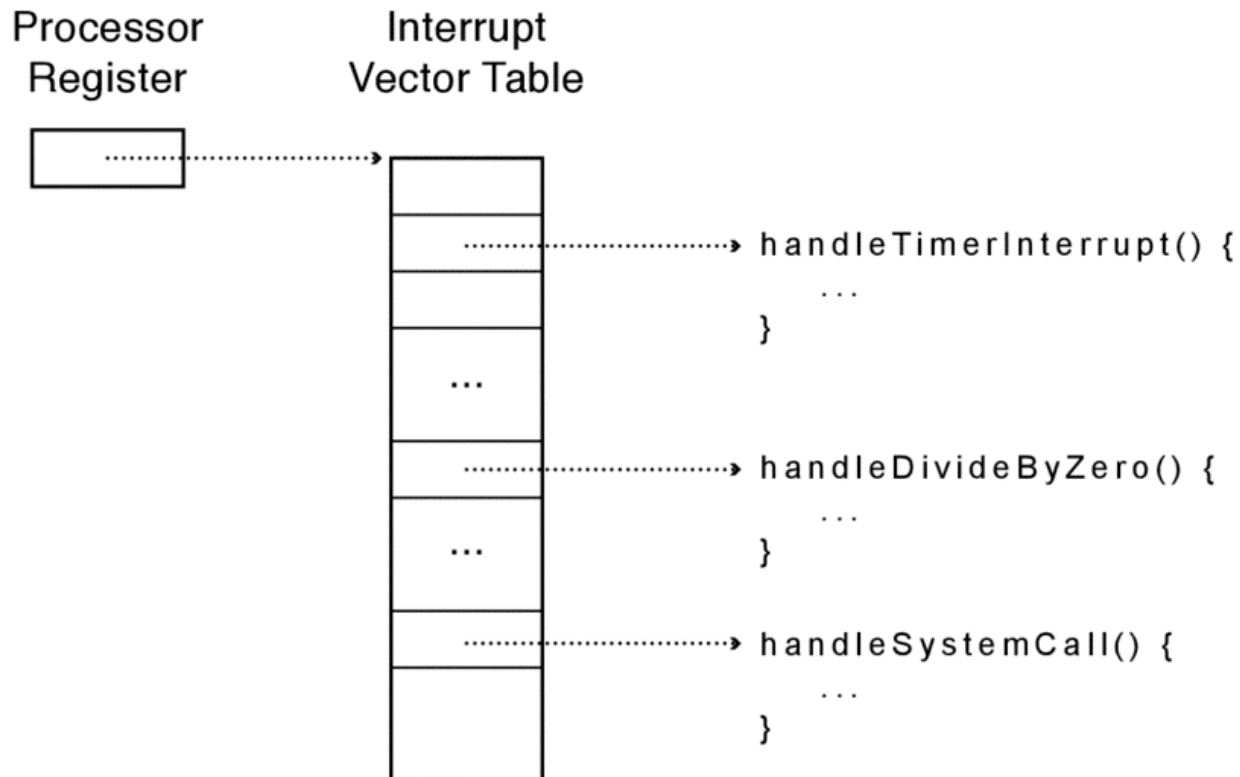
- From kernel-mode to user-mode
 - Return from interrupt, exception, system call
 - Resume suspended execution
 - New process/new thread start
 - Jump to first instruction in program/thread
 - Process/thread context switch
 - Resume some other process
 - User-level upcall
 - Asynchronous notification to user program

How do we take interrupts safely?

- Interrupt vector
 - Limited number of entry points into kernel
- Kernel interrupt stack
 - Handler works regardless of state of user code
- Interrupt masking
 - Handler is non-blocking
- Atomic transfer of control
 - Single instruction to change:
 - Program counter
 - Stack pointer
 - Memory protection
 - Kernel/user mode
- Transparent restartable execution
 - User program does not know interrupt occurred

Interrupt Vector

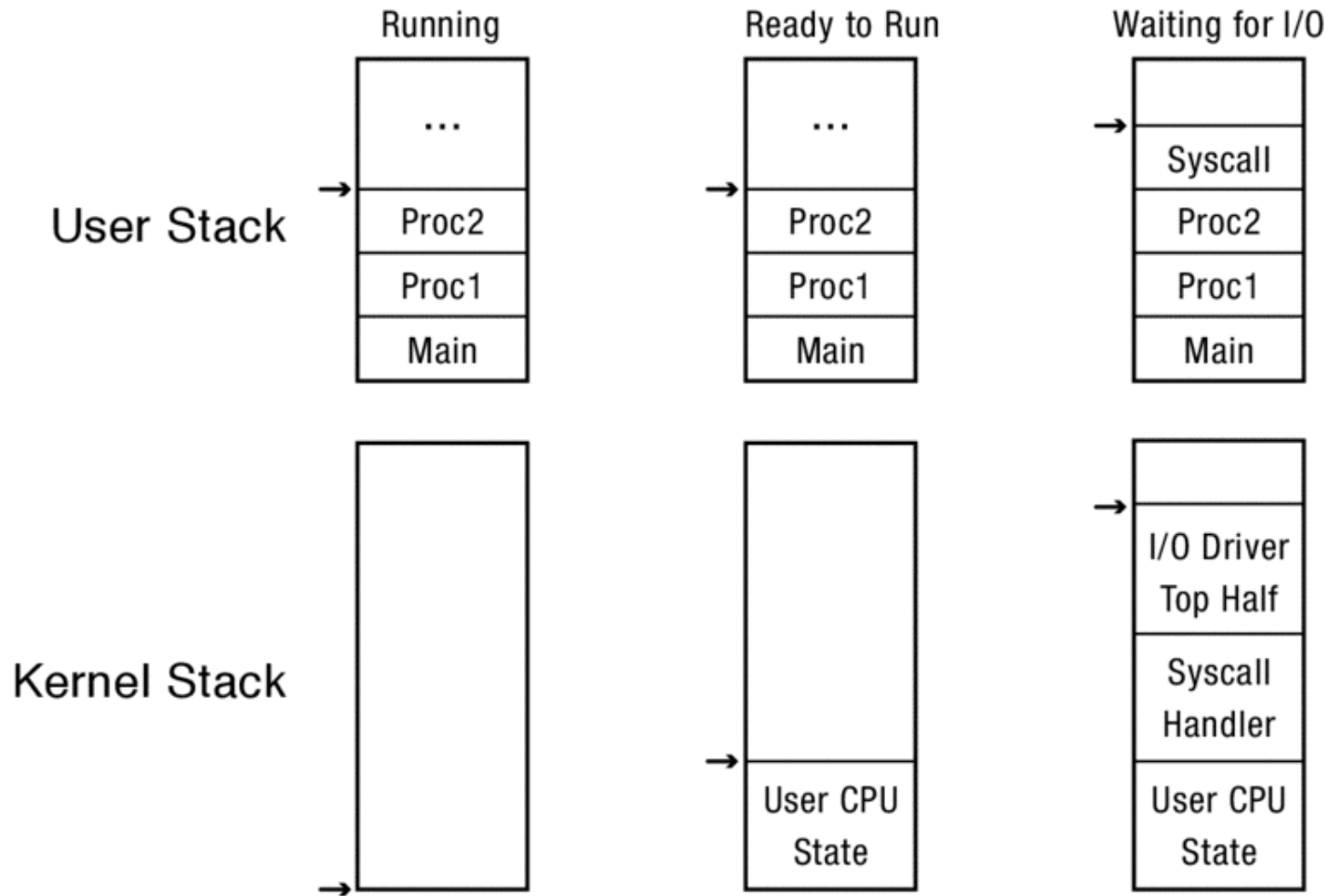
- Table set up by OS kernel; pointers to code to run on different events



Interrupt Stack

- Per-processor, located in kernel (not user) memory
 - Usually a thread has both: kernel and user stack
- Why can't interrupt handler run on the stack of the interrupted user process?

Interrupt Stack



Interrupt Masking

- Interrupt handler runs with interrupts off
 - Re-enabled when interrupt completes
- OS kernel can also turn interrupts off
 - Eg., when determining the next process/thread to run
 - If defer interrupts too long, can drop I/O events
 - On x86
 - CLI: disable interrupts
 - STI: enable interrupts
 - Only applies to the current CPU
- Cf. implementing synchronization, chapter 5

Interrupt Handlers

- Non-blocking, run to completion
 - Minimum necessary to allow device to take next interrupt
 - Any waiting must be of limited duration
 - Wake up other threads to do any real work
- Rest of device driver runs as a kernel thread
 - Queues work for interrupt handler
 - (Sometimes) wait for interrupt to occur

Atomic Mode Transfer

- On interrupt (x86)
 - Save current stack pointer
 - Save current program counter
 - Save current processor status word (condition codes)
 - Switch to kernel stack; put SP, PC, PSW on stack
 - Switch to kernel mode
 - Vector through interrupt table
 - Switch to handler, PC & kernel PSW
 - Interrupt handler saves registers it might clobber
 - Interrupt management

in hardware

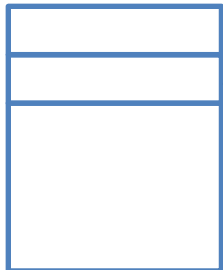
Before

User-level
process

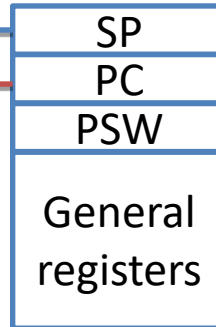
Code:

```
Foo() {  
  while(...) {  
    x=x+1;  
    y=y-2;  
  }  
}
```

Stack:



Registers: (x86 names):



SS:ESP

CS:EIP

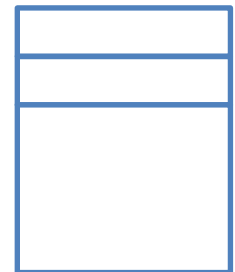
EFLAGS

Kernel

Code:

```
handler() {  
  push A  
  ...  
}
```

Exception
Stack:



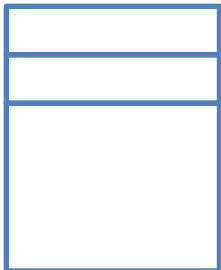
During

User-level
process

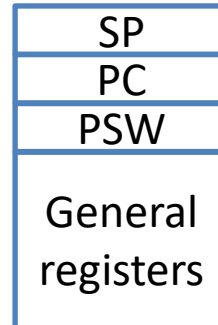
Code:

```
Foo() {  
  while(...) {  
    x=x+1;  
    y=y-2;  
  }  
}
```

Stack:



Registers:



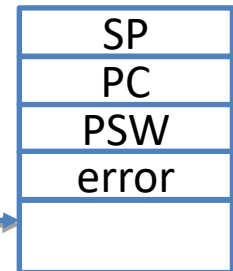
Kernel

Code:

```
handler() {  
  pusha  
  ...  
  popa  
  IRET  
}
```

Exception

Stack:



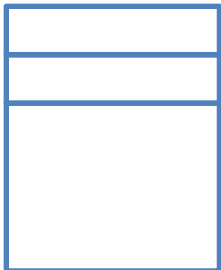
During

User-level
process

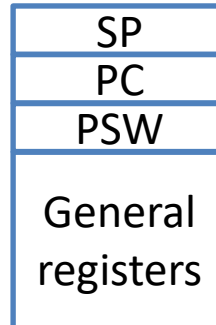
Code:

```
Foo() {  
  while(...) {  
    x=x+1;  
    y=y-2;  
  }  
}
```

Stack:



Registers:



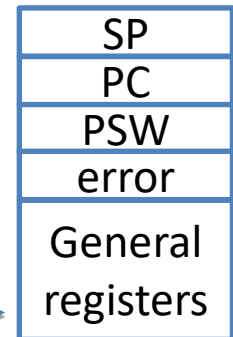
Kernel

Code:

```
handler() {  
  pusha  
  ...  
  popa  
  IRET  
}
```

Exception

Stack:



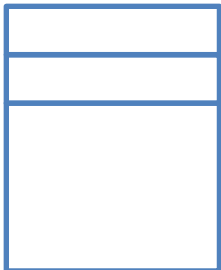
After

User-level
process

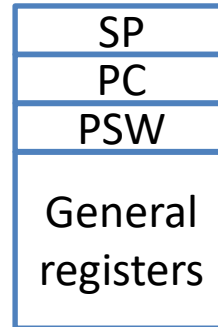
Code:

```
Foo() {  
    while(...) {  
        x=x+1;  
        y=y-2;  
    }  
}
```

Stack:



Registers:

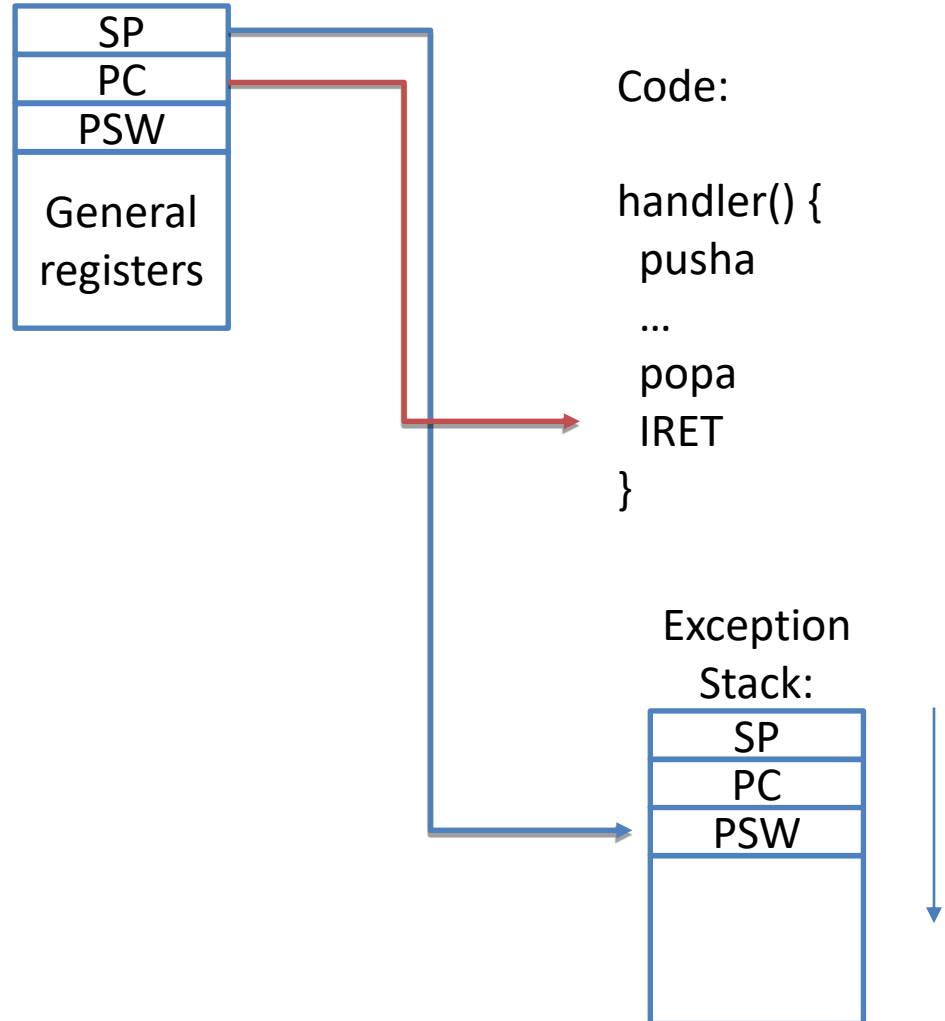
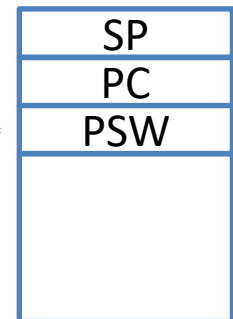


Kernel

Code:

```
handler() {  
    pusha  
    ...  
    popa  
    IRET  
}
```

Exception
Stack:



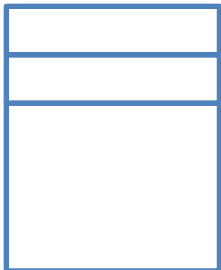
... and after

User-level
process

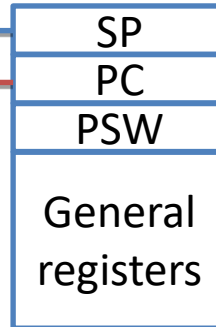
Code:

```
Foo() {  
  while(...) {  
    x=x+1;  
    y=y-2;  
  }  
}
```

Stack:



Registers:

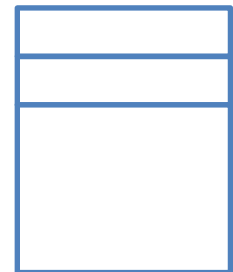


Kernel

Code:

```
handler() {  
  pusha  
  ...  
  popa  
  IRET  
}
```

Exception
Stack:



At end of handler

- Handler restores saved registers
- Atomically return to interrupted process/thread (IRET instruction in the previous example)
 - Restore program counter
 - Restore program stack
 - Restore processor status word/condition codes
 - Switch to user mode
 - Enable interrupts

A (very) simplified architecture

- Let's suppose a 16-bit processor with a few general registers **R1..Rn**, and three status and control registers **PC**, **SP**, and **PS**
- One single kernel stack for managing interrupts
- Interrupt checked at the end of the fetch-execute cycle (PC already incremented)
- IRET instruction to return from the interrupt handler. It pops PC, SP, and PSW out from the kernel stack and restores them atomically!

Interrupt management: a simplified example

Initial state: interrupt '500' occurs when executing instruction A000

Registri nella CPU

PC	A000
PS	PSW P
SP	FFFC
R1	AAAA
R2	BBBB
...	

Memoria

Programma P

...	...
A000	istr. 1
A004	istr. 2
A008	istr. 3
A00C	istr. 4
A010	istr. 5
...	

Interrupt Handler

100	Store
104	Save
108	Registers
...	
200	Restores
204	Registers
208	IRET

Stack di P

...	...
FFEC	
FFF0	
FFF4	
FFF8	
FFFC	...

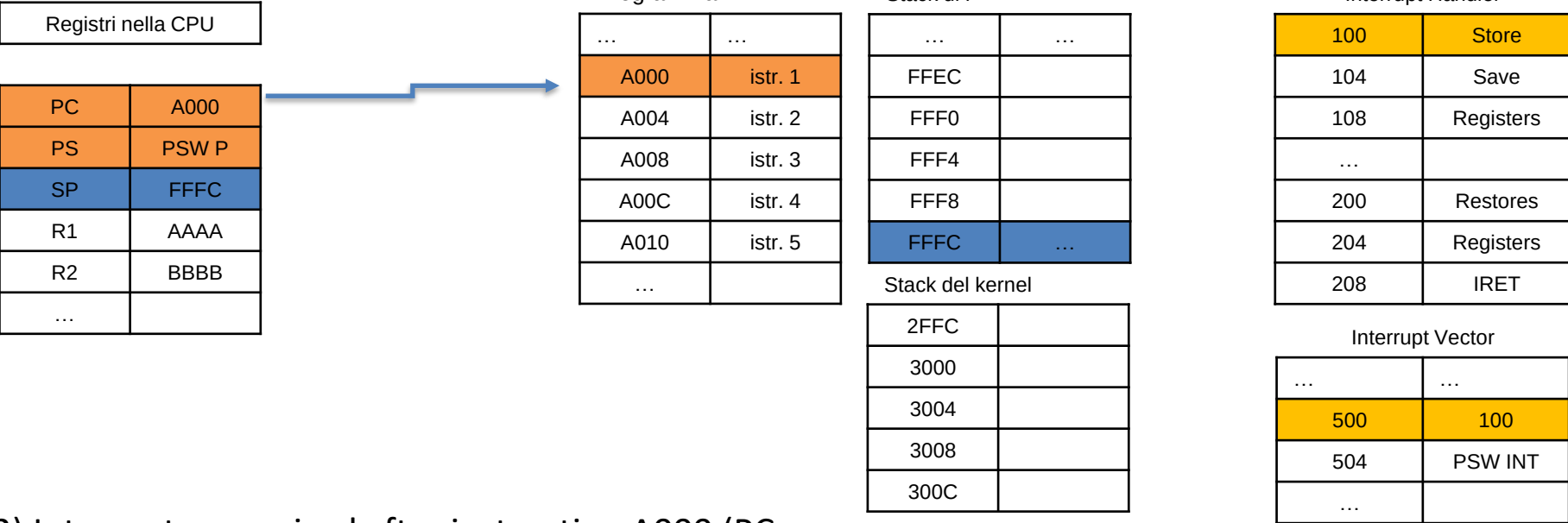
Stack del kernel

2FF8	
2FFC	
3000	
3004	
3008	
300C	

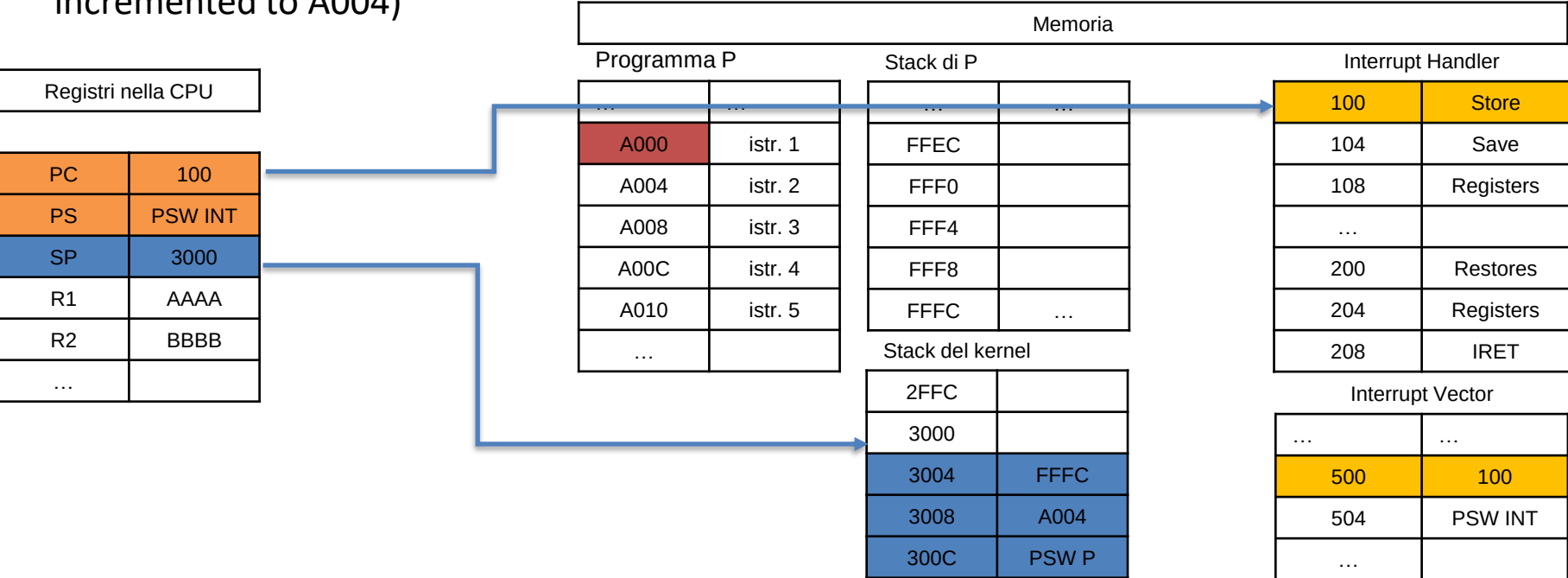
Interrupt Vector

...	...
500	100
504	PSW INT
...	

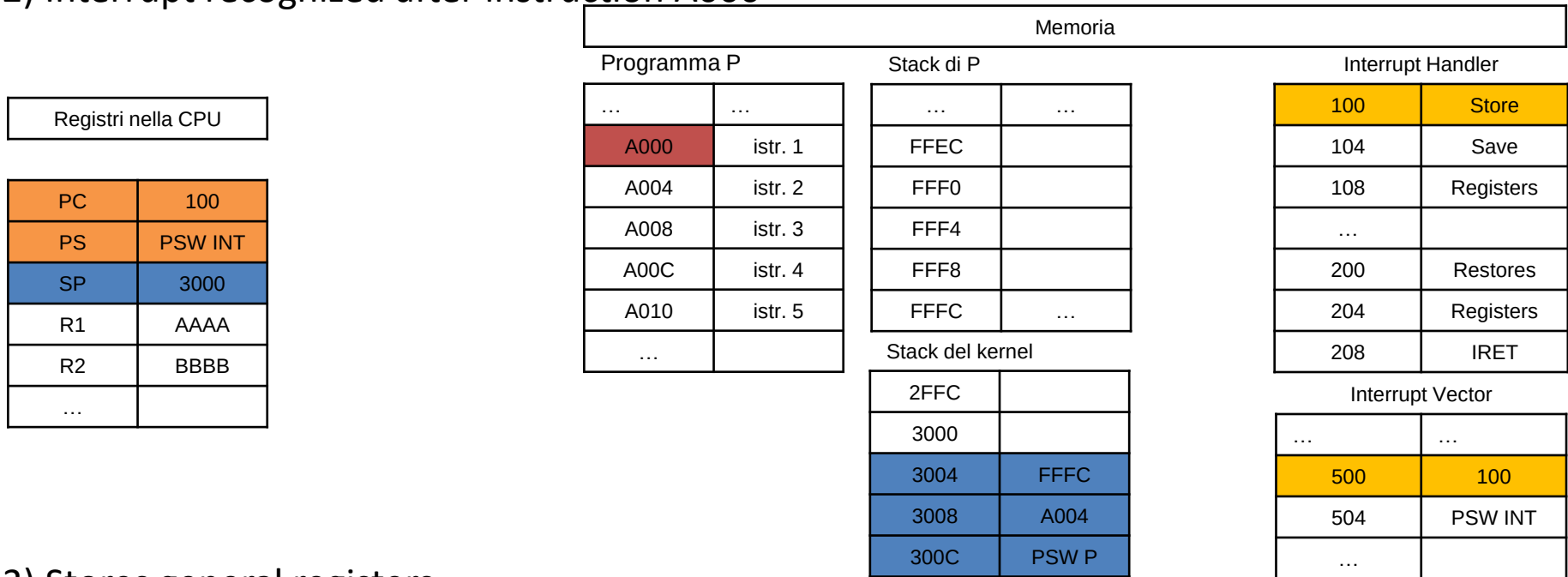
1) Initial state



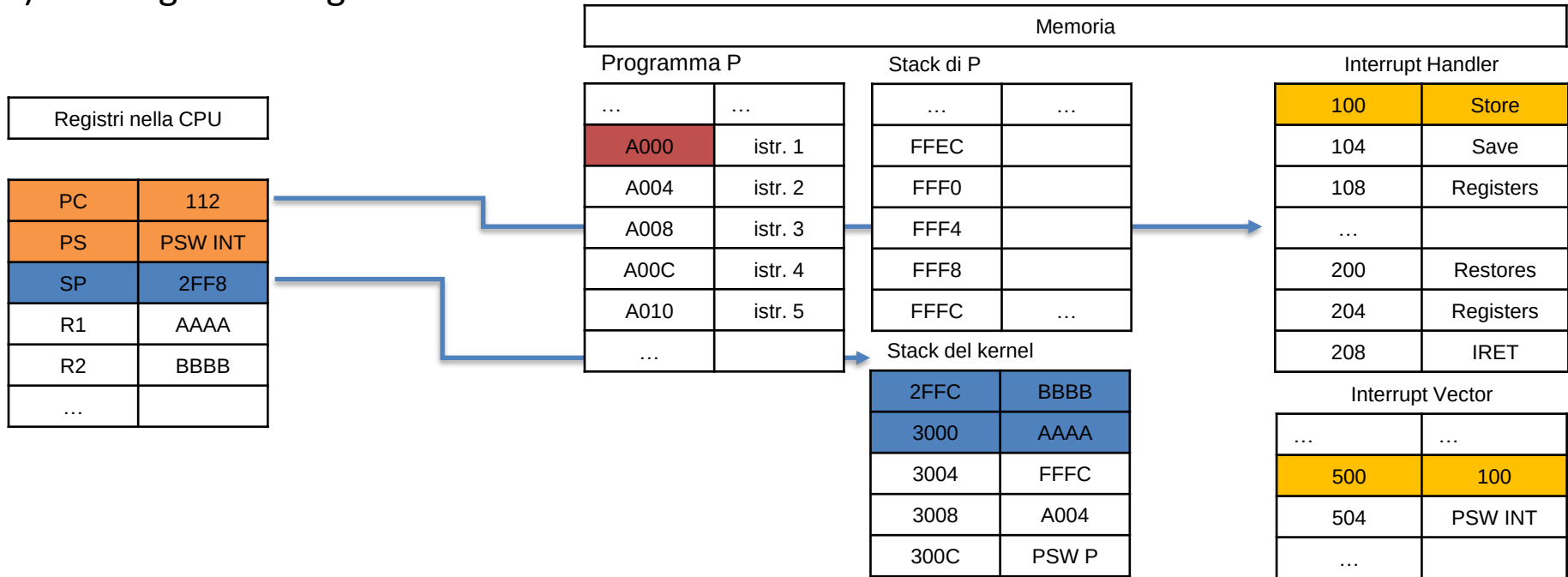
2) Interrupt recognized after instruction A000 (PC incremented to A004)



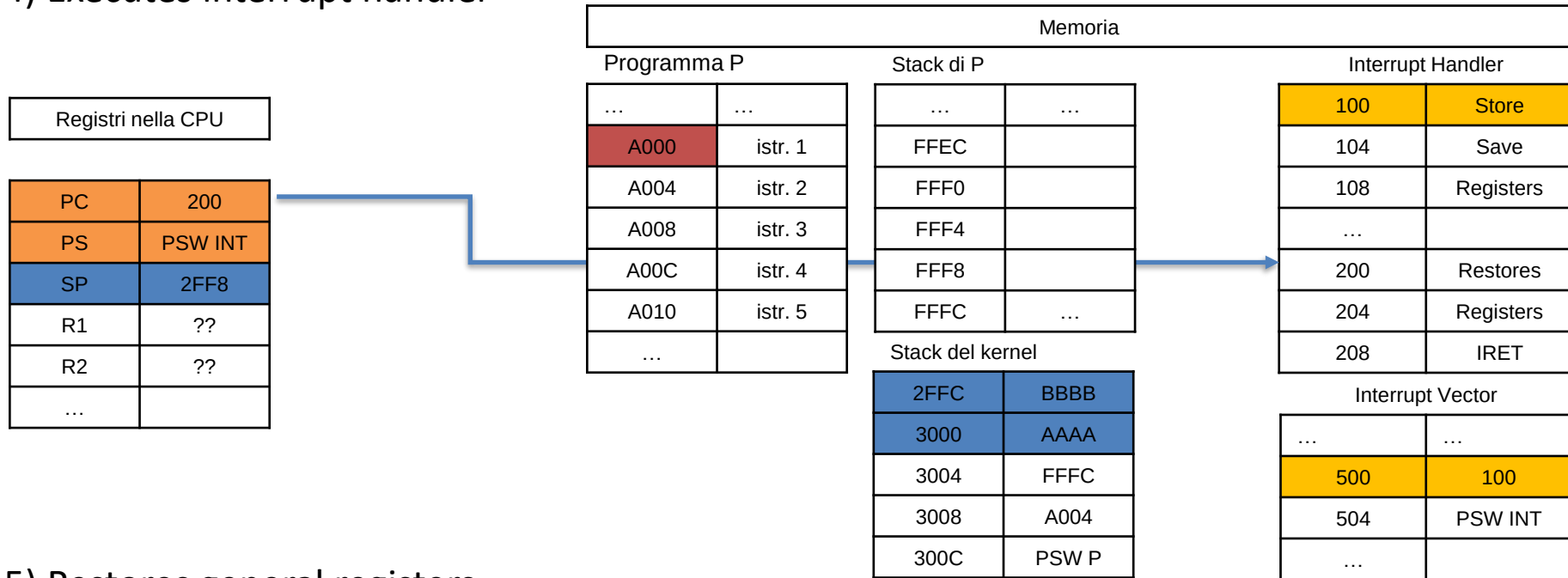
2) Interrupt recognized after instruction A000



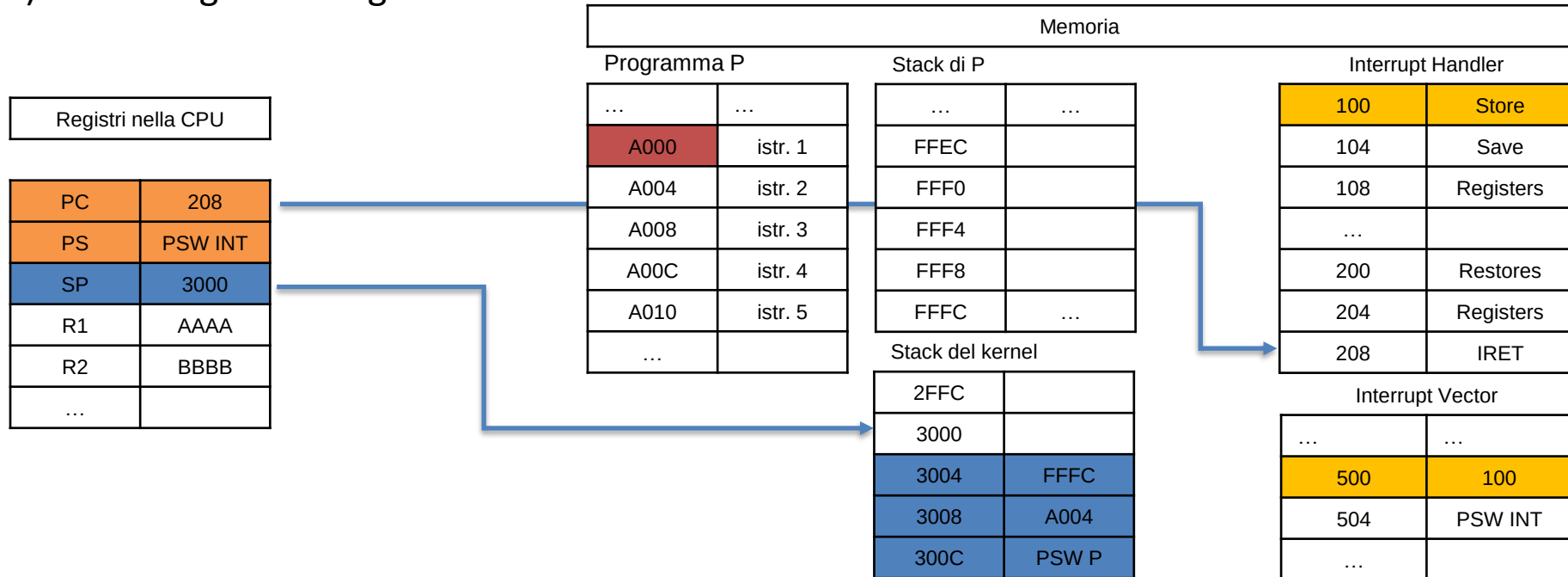
3) Stores general registers



4) Executes interrupt handler



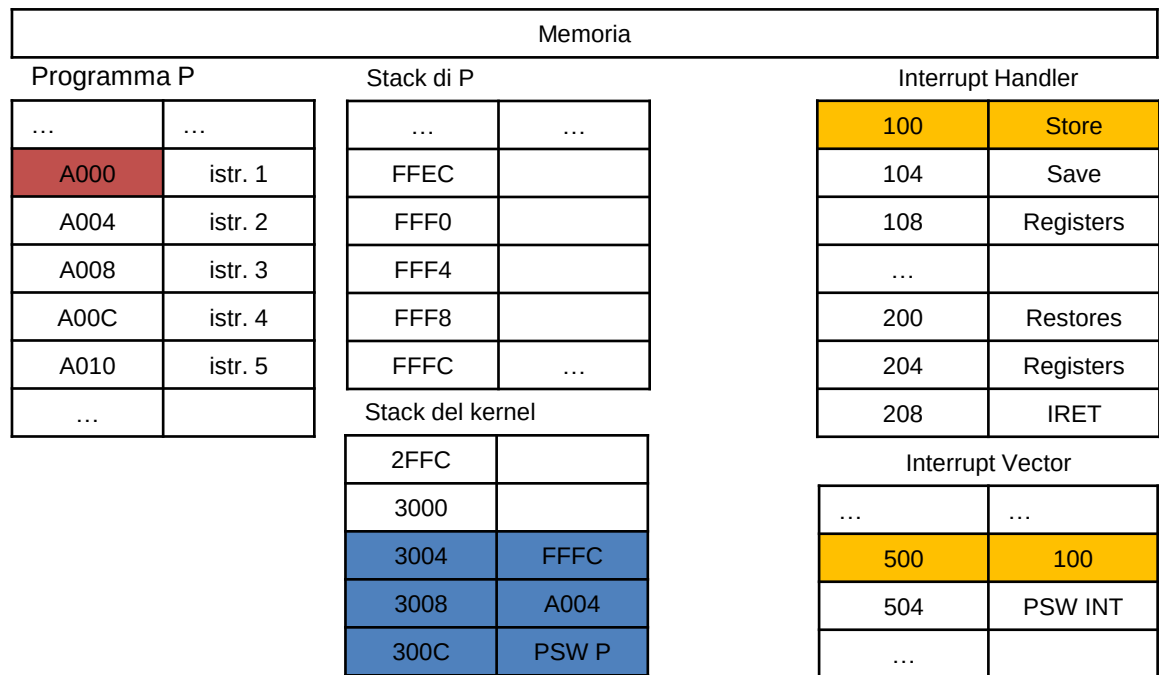
5) Restores general registers



5) Restores general registers

Registri nella CPU

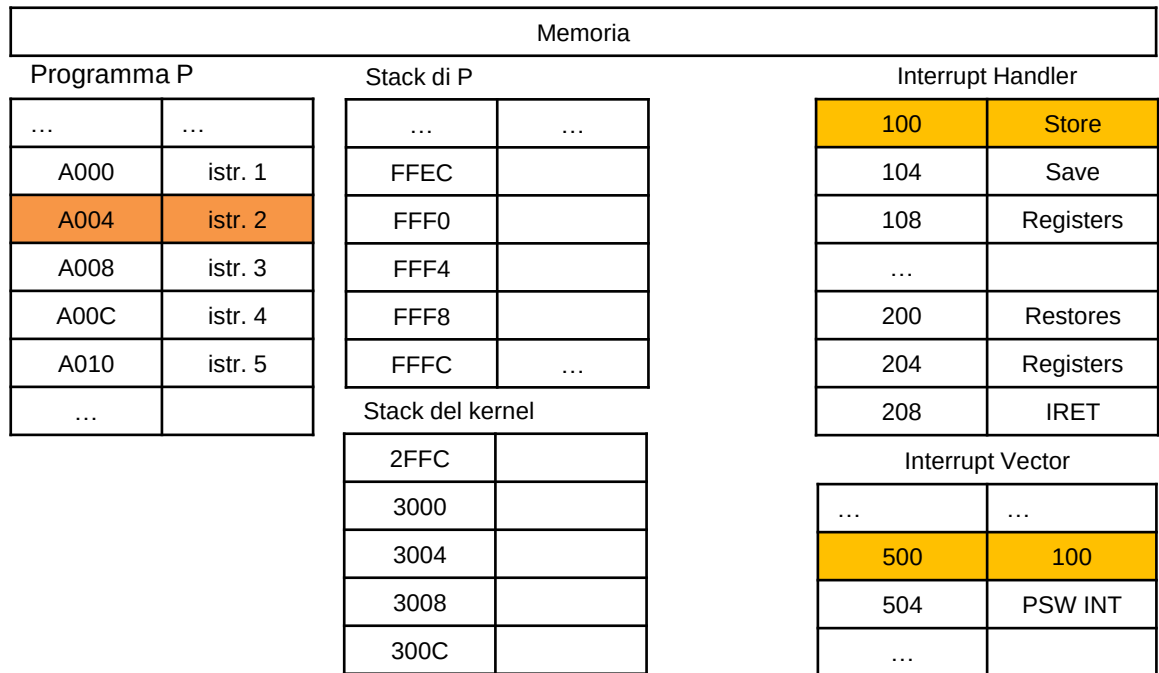
PC	208
PS	PSW INT
SP	3000
R1	AAAA
R2	BBBB
...	



6) Executes IRET

Registri nella CPU

PC	A004
PS	PSW P
SP	FFFC
R1	AAAA
R2	BBBB
...	

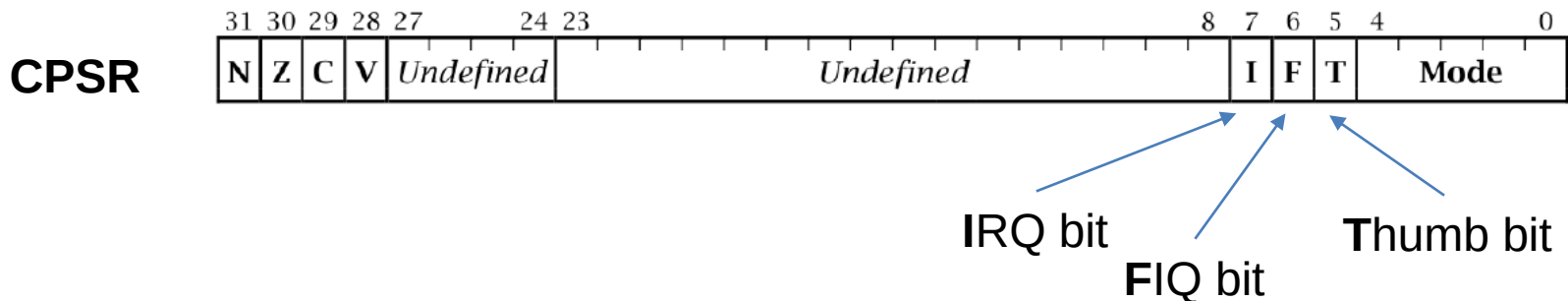


Exception handling in ARM

- ARM processor has 6 modes of operation
- Switching between modes can be done by modifying the mode bits in the CPSR register
- User-mode operates at PL0, the other modes at PL1
- All system resources can be accessed in PL1 mode
- In PL1 mode, some banked registers are used to execute the exception handler

Exception mode and CPSR

Mode	CPSR _{4:0}
User	10000
Supervisor	10011
Abort	10111
Undefined	11011
Interrupt (IRQ)	10010
Fast Interrupt (FIQ)	10001



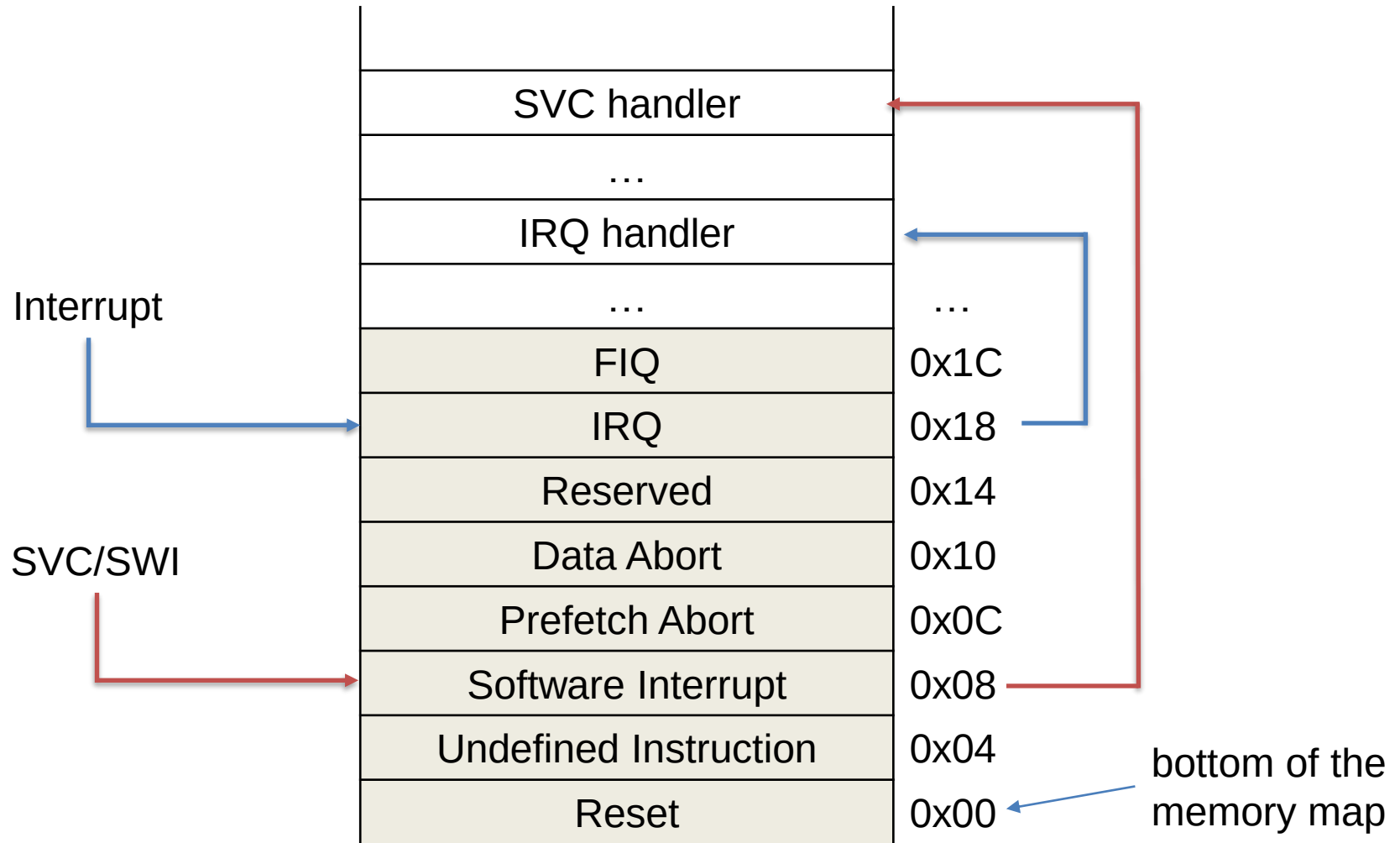
Exception vector table

Exception	Address	Mode
Reset	0x00	Supervisor
Undefined Instruction	0x04	Undefined
Supervisor Call	0x08	Supervisor
Prefetch Abort (instruction fetch error)	0x0C	Abort
Data Abort (data load or store error)	0x10	Abort
Reserved	0x14	N/A
Interrupt	0x18	IRQ
Fast Interrupt	0x1C	FIQ

Priority (1 high, 6 low):

- 1 Reset
- 2 Data Abort
- 3 FIQ
- 4 IRQ
- 5 Prefetch Abort
- 6 Undefined Instruction and Software Interrupt (SWI/SVC)

Exception vector table



- Each entry has only 32 bits and contains: a ***branch instruction*** or ***load PC instruction*** to the actual handler

Banked registers

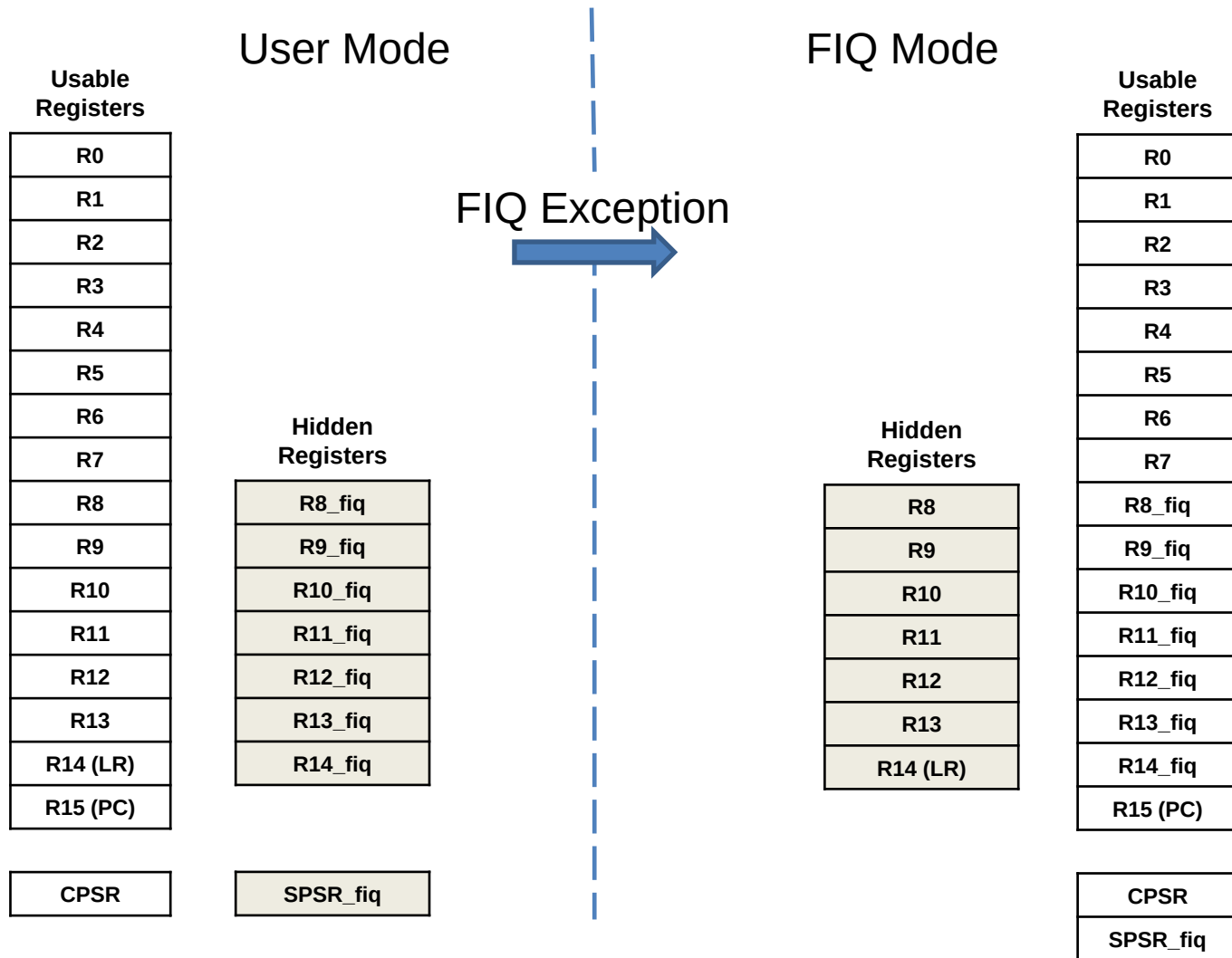
User	System	Fast Interrupt	Interrupt	Supervisor	Abort	Undefined
R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7
R8	R8	R8_fiq	R8	R8	R8	R8
R9	R9	R9_fiq	R9	R9	R9	R9
R10	R10	R10_fiq	R10	R10	R10	R10
R11	R11	R11_fiq	R11	R11	R11	R11
R12	R12	R12_fiq	R12	R12	R12	R12
R13 (SP)	R13 (SP)	R13_fiq	R13_irq	R13_svc	R13_abt	R13_und
R14 (LR)	R14 (LR)	R14_fiq	R14_irq	R14_svc	R14_abt	R14_und
R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)	R15 (PC)

Program Status Registers

CPSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
		SPSR_fiq	SPSR_irq	SPSR_svc	SPSR_abt	SPSR_und

- 37 registers in total (31 general purpose, 6 control registers)
- In PL1, SP LR and CPSR are banked; FIQ mode has 5 additional banked registers

Visibility of banked registers: FIQ example



Response to an Exception

- Set the banked LR register to the return address (e.g., PC+4)
- Copies the CPSR into the banked SPSR
 - This save the current mode, the interrupt mask and condition flags
- Change the appropriate CPSR mode bits
- Map in the appropriate banked registers for that mode
 - For FIQ mode there are 5 extra banked registers
- Disable interrupts
 - *IRQs are disabled when any exception occurs*
 - FIQs are disabled when FIQ occurs, and *on reset*
- Set the PC to the vector address for the exception handler

Exception management (some cases)

Reset:

```
R14_rst = PC
SPSR_rst = CPSR
CPSR[4:0] = 0x10011 // supervisor mode
CPSR[5] = 0 // ARM state
CPSR[6] = 1 // Disable FIQ
CPSR[7] = 1 // Disable IRQ
PC = 0x00000000
```

IRQ:

```
R14_irq = PC+4
SPSR_irq = CPSR
CPSR[4:0] = 0x10010 // IRQ mode
CPSR[5] = 0 // ARM state
// CPSR[6] unchanged
CPSR[7] = 1 // Disable IRQ
PC = 0x00000018
```

SVC:

```
R14_svc = PC+4 // next instruction
SPSR_svc = CPSR
CPSR[4:0] = 0x10011 // supervisor mode
CPSR[5] = 0 // ARM state
// CPSR[6] unchanged
CPSR[7] = 1 // Disable IRQ
PC = 0x00000008
```

FIQ:

```
R14_fiq = PC+4
SPSR_fiq = CPSR
CPSR[4:0] = 0x10001 // FIQ mode
CPSR[5] = 0 // ARM state
CPSR[6] = 1 // Disable FIQ
CPSR[7] = 1 // Disable IRQ
PC = 0x0000001C
```


Returning from an Exception

- In general, the steps are:
 - Restore the CPSR from the banked SPSR, which restores the execution mode and privilege level
 - Restore the PC from the banked LR
- The above two steps must be simultaneous (i.e., *atomic*)!
- The exception return is carried out by a *data processing instruction* with the 'S' flag set, specifically **MOVS** and **SUBS**
 - For example (**if not using the stack**):
 - **MOVS PC, R14_svc** // pc = lr_svc ; cpsr = spsr_svc
 - **SUBS PC, R14_fiq, #4** // pc = lr_fiq-4 ; cpsr = spsr_fiq

Returning from an Exception

- Returning from *SVC*
 - Exception generated by the instruction itself; PC not updated; LR banked already contains PC+4, i.e., next instruction to execute
 - If not using the stack: **MOVS PC, LR_svc**
 - If using the stack:

STMFD SP!, {reglist, LR} // handler entry

....

LDMFD SP!, {reglist, PC}^ // handler exit

(*) The ^ qualifier specifies that the CPSR is restored from the SPSR. It must be used only from privileged modes

Returning from an Exception

- Returning from FIQ and IRQ
 - Interrupt received only after the execution of an instruction, thus PC already updated to point to the next instruction; LR banked contains PC+4, but PC instruction has not been executed. Therefore, LR must be decremented by 4
 - If not using the stack: **SUBS PC, LR_fiq/irq, #4**
 - If using the stack:

SUB LR, LR, #4 // handler entry

STMFD SP!, {reglist, LR}

....

LDMFD SP!, {reglist, PC}^ // handler exit

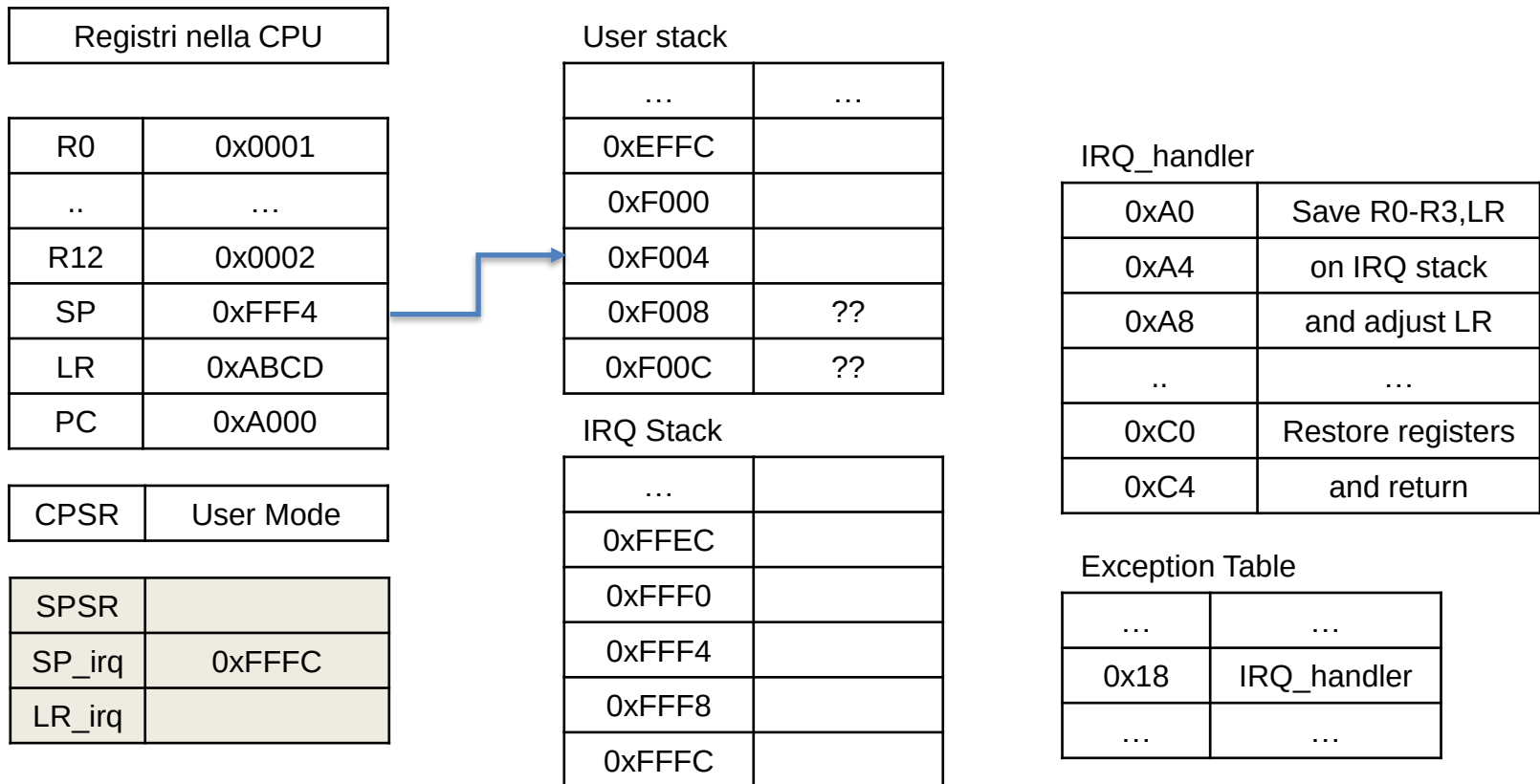
Returning from an Exception

Summary

	Return instruction	ARM LR_x	THUMB LR_x
SWI	MOVS PC, LR_svc	PC+4	PC+2
UNDEF	MOVS PC, LR_und	PC+4	PC+2
FIQ	SUBS PC, LR_fig, #4	PC+4	PC+4
IRQ	SUBS PC, LR_irq, #4	PC+4	PC+4
PrefetchABT	SUBS PC, LR_abt, #4	PC+4	PC+4
DataABT	SUBS PC, LR_abt, #8	PC+8	PC+8
RESET	-	?	?

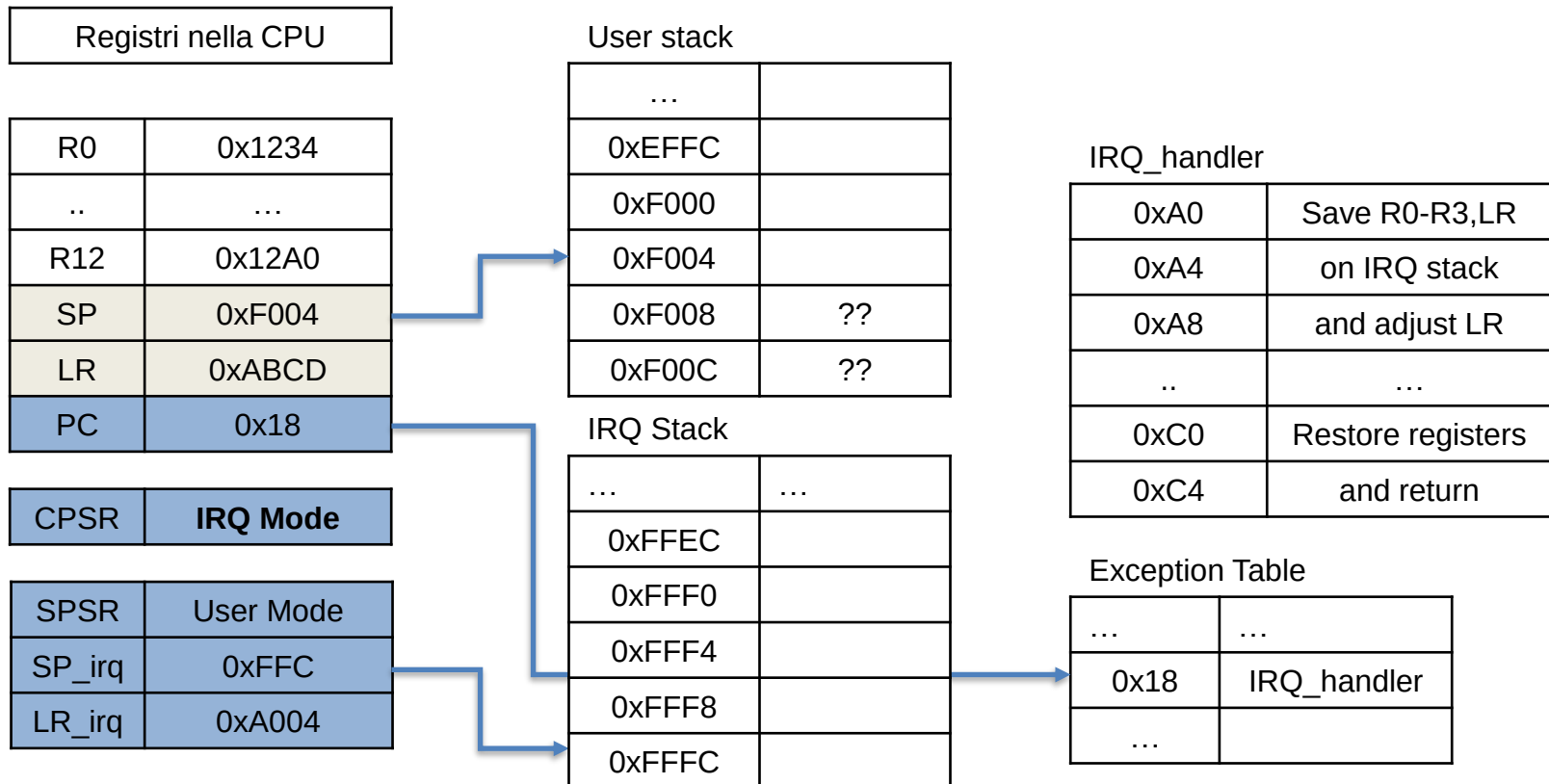
Interrupt management example

Step1: Initial state, IRQ occurs when executing instruction 0xA000



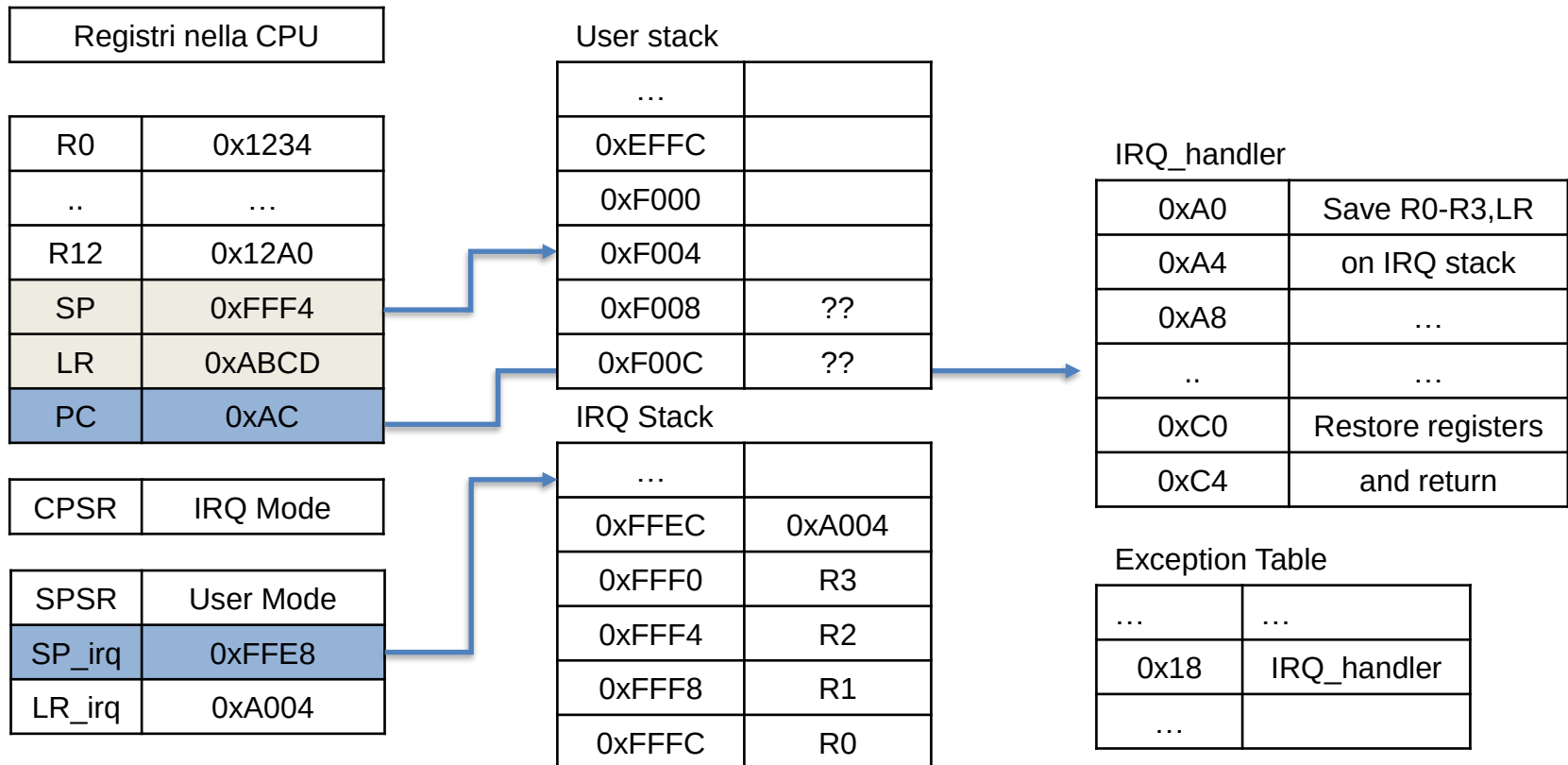
Interrupt management example

Step2: Switching to IRQ Mode, map to banked SP and LR and be prepared to execute the IRQ handler



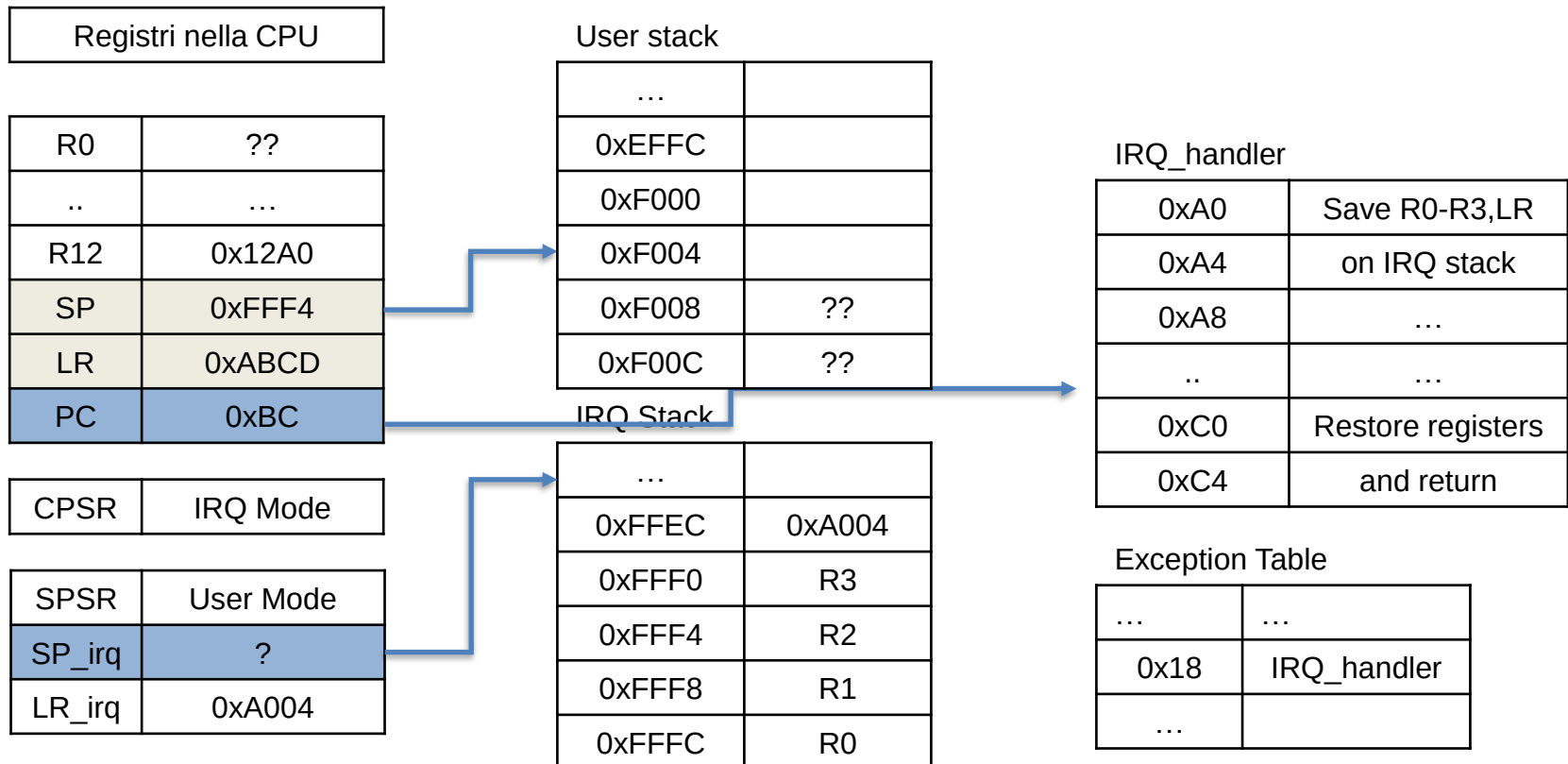
Interrupt management example

Step3: Start executing top-level handler for IRQ



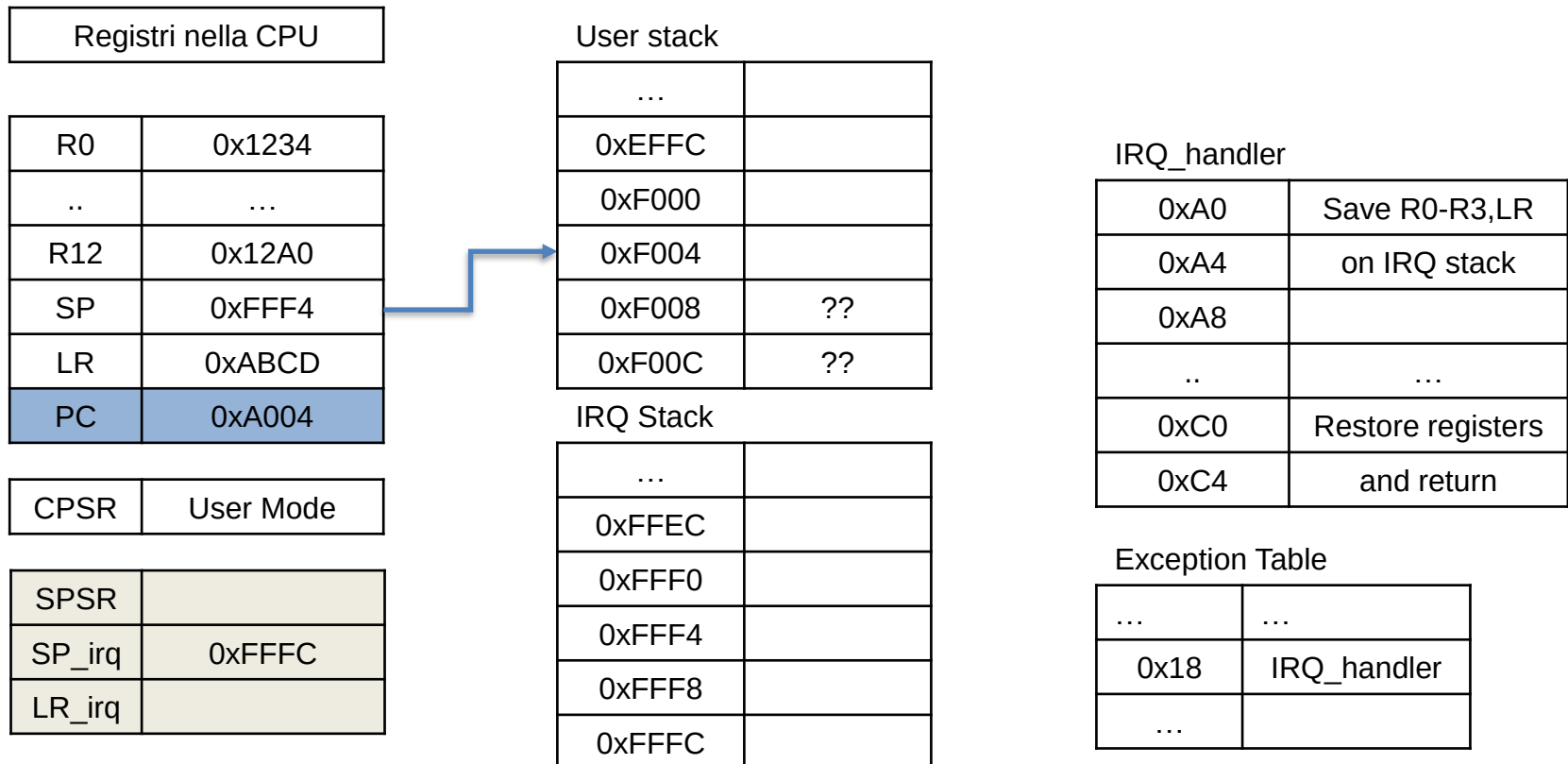
Interrupt management example

Step4: IRQ handled, now we have to restore the state before the IRQ



Interrupt management example

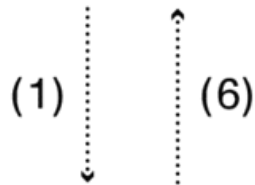
Step4: Restore user registers and switch to User Mode, LR_irq (on the stack) will be copied into



Software Interrupts aka System Calls

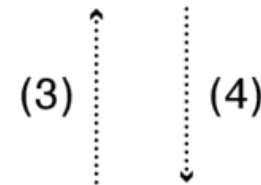
User Program

```
main () {  
    file_open(arg1, arg2);  
}
```



Kernel

```
file_open(arg1, arg2) {  
    // do operation  
}
```



User Stub

```
file_open(arg1, arg2) {  
    push #SYSCALL_OPEN  
    trap  
    return  
}
```

(2)

Hardware Trap

Kernel Stub

```
file_open_handler() {  
    // copy arguments  
    // from user memory  
    // check arguments  
    file_open(arg1, arg2);  
    // copy return value  
    // into user memory  
    return;  
}
```

Trap Return

(5)

This by a single
process instruction.
For example: SVC
(supervisor call) or
TRAP

This by IRET-equivalent
instruction

Kernel System Call Handler

- Locate arguments
 - In registers or on user(!) stack
- ***Copy arguments***
 - From user memory into kernel memory
 - Protect kernel from malicious code evading checks
- **Validate arguments**
 - Protect kernel from errors in user code
- Copy results back
 - into user memory

System Calls in ARM Linux

- The system call handler must know which kind of software interrupt is being called (i.e., SVC number) and where to find the arguments
- *man 2 syscall*

Arch/ABI	Instruction	System call #	Ret val	Ret val2	Error
alpha	callsys	v0	v0	a4	a3
arc	trap0	r8	r0	-	-
arm/OABI	swi NR	-	a1	-	-
arm/EABI	swi 0x0	r7	r0	r1	-
arm64	svc #0	x8	x0	x1	-
blackfin	excpt 0x0	P0	R0	-	-
i386	int \$0x80	eax	eax	edx	-

Arch/ABI	arg1	arg2	arg3	arg4	arg5	arg6	arg7
alpha	a0	a1	a2	a3	a4	a5	-
arc	r0	r1	r2	r3	r4	r5	-
arm/OABI	a1	a2	a3	a4	v1	v2	v3
arm/EABI	r0	r1	r2	r3	r4	r5	r6
arm64	x0	x1	x2	x3	x4	x5	-
blackfin	R0	R1	R2	R3	R4	R5	-
i386	ebx	ecx	edx	esi	edi	ebp	-

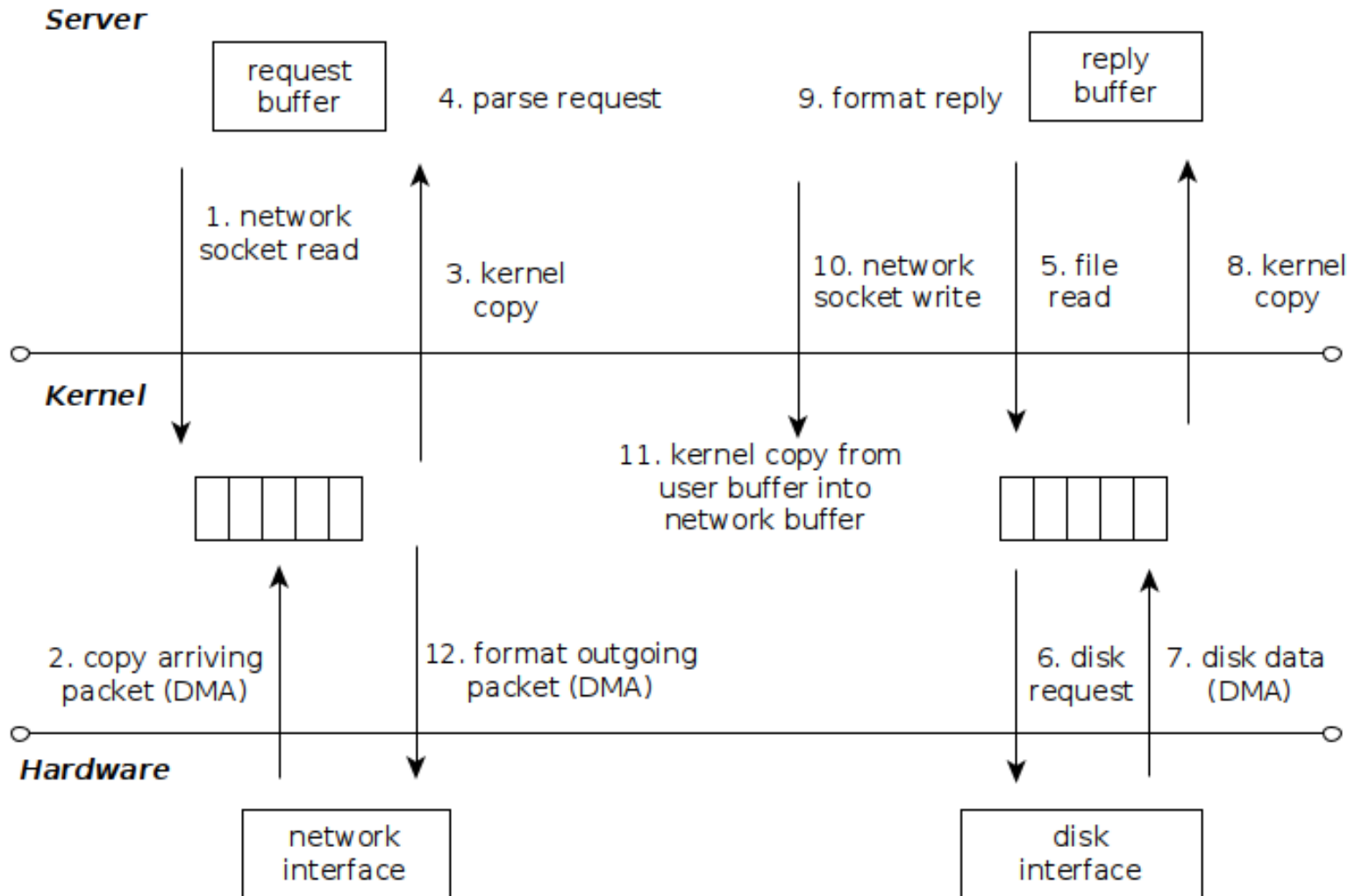
Top-Level SWI handler

[illegible]

Do applications use System Calls?

- Yes, they do!
- Reading/Writing to disks, network cards, screens,, all these ops require system calls.
- Many daily-life applications make large use of system calls
 - For example: a web server
 - Other examples?

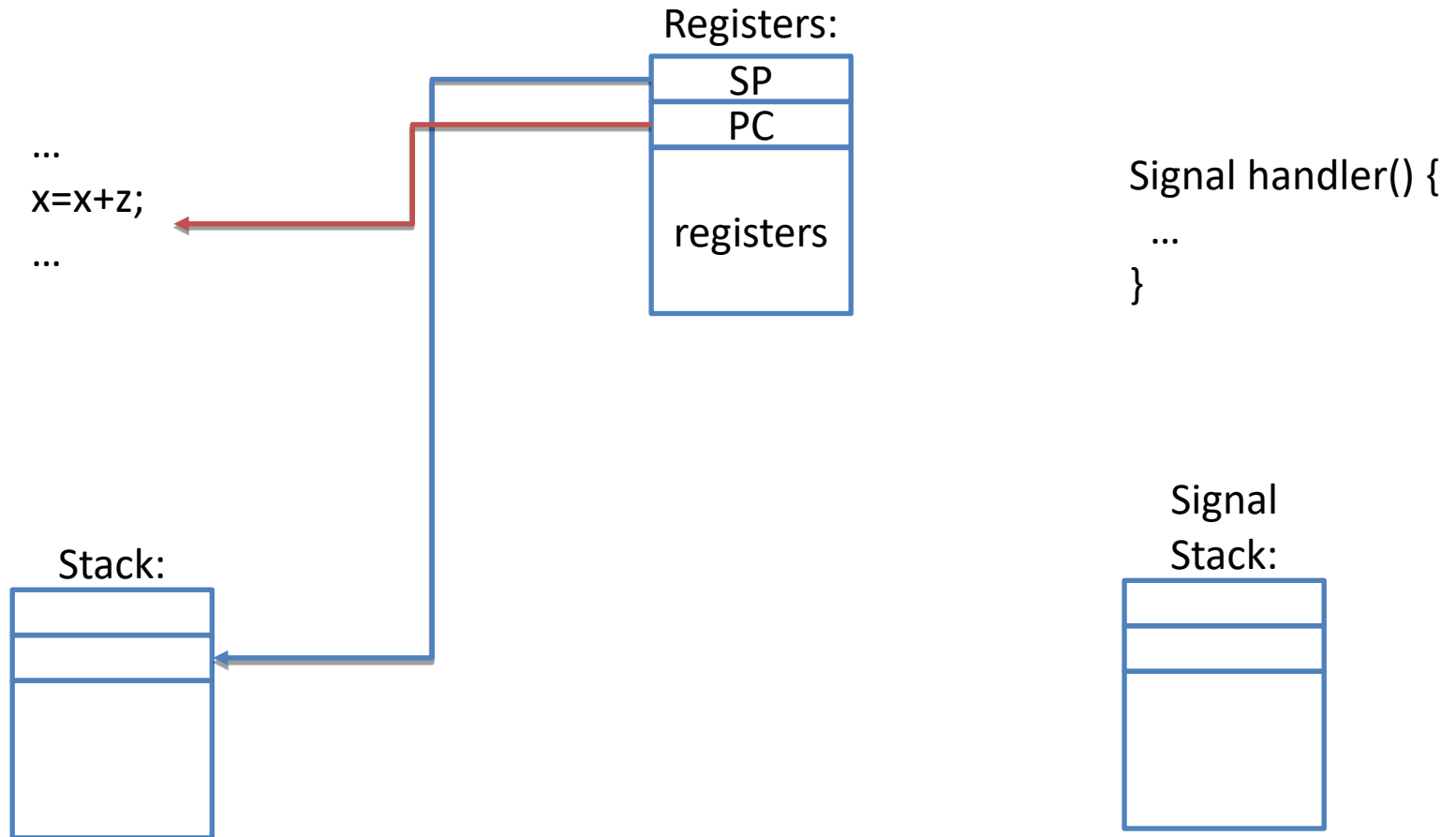
Web Server Example



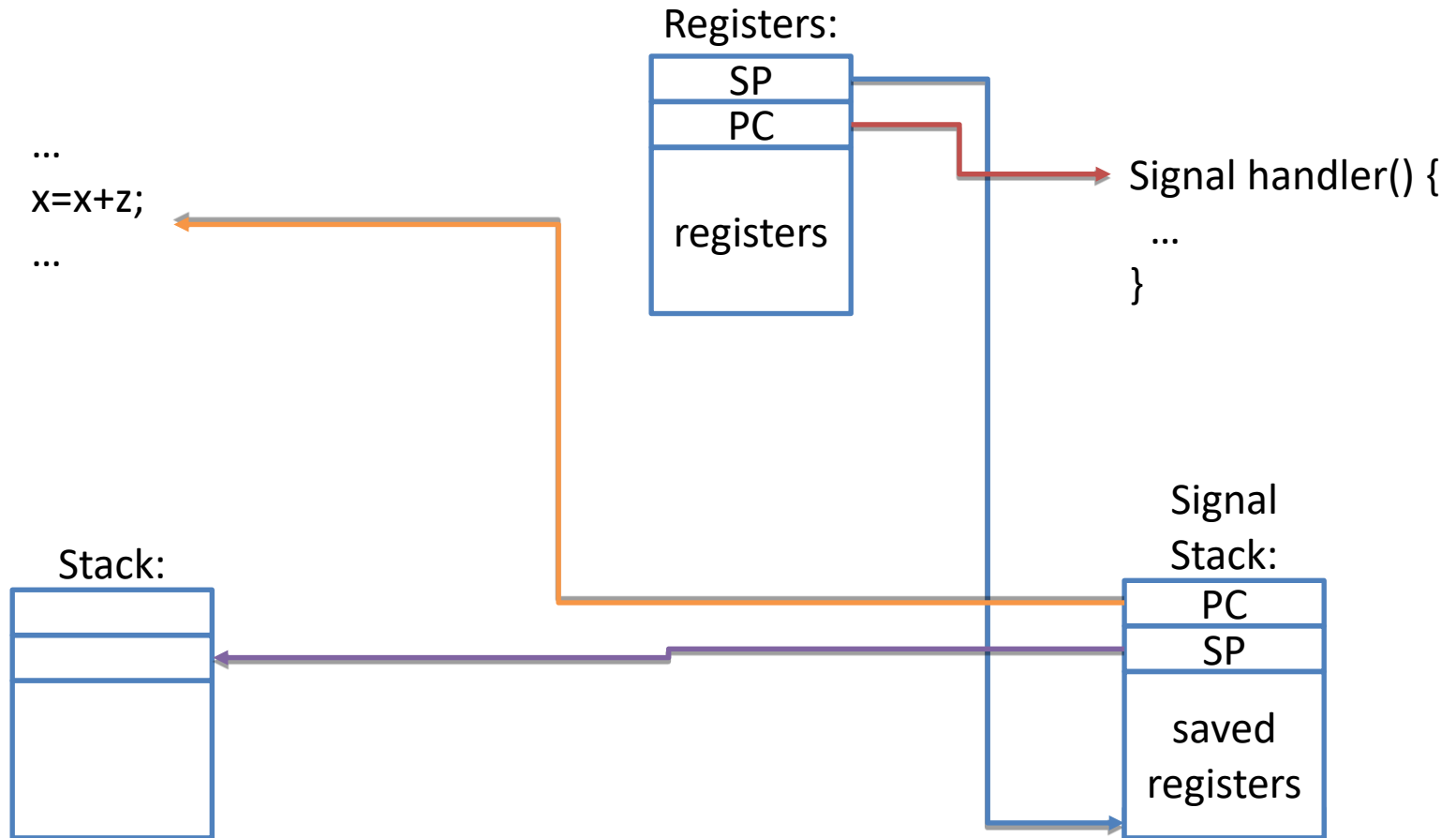
Upcall: User-level interrupt

- Also known as (aka) **UNIX *signals***
 - Notify user process of event that needs to be handled right away
 - Time-slice for user-level thread manager
 - Interrupt delivery for VM player (see later)
- Direct analogue of kernel interrupts
 - Signal handlers – fixed entry points
 - Separate signal stack
 - Automatic save/restore registers – transparent resume
 - Signal masking: signal disabled while in signal handler

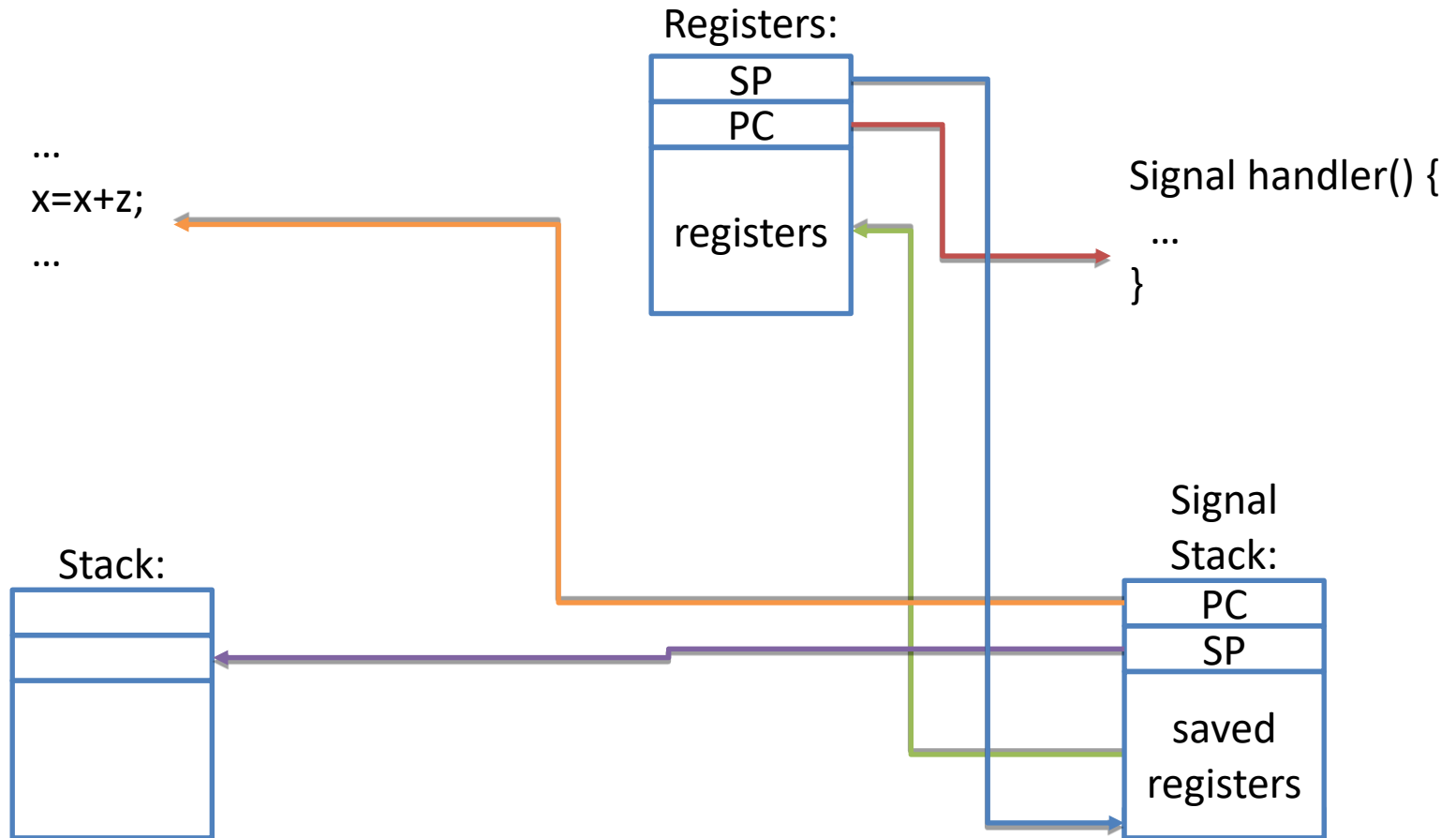
Upcall: Before



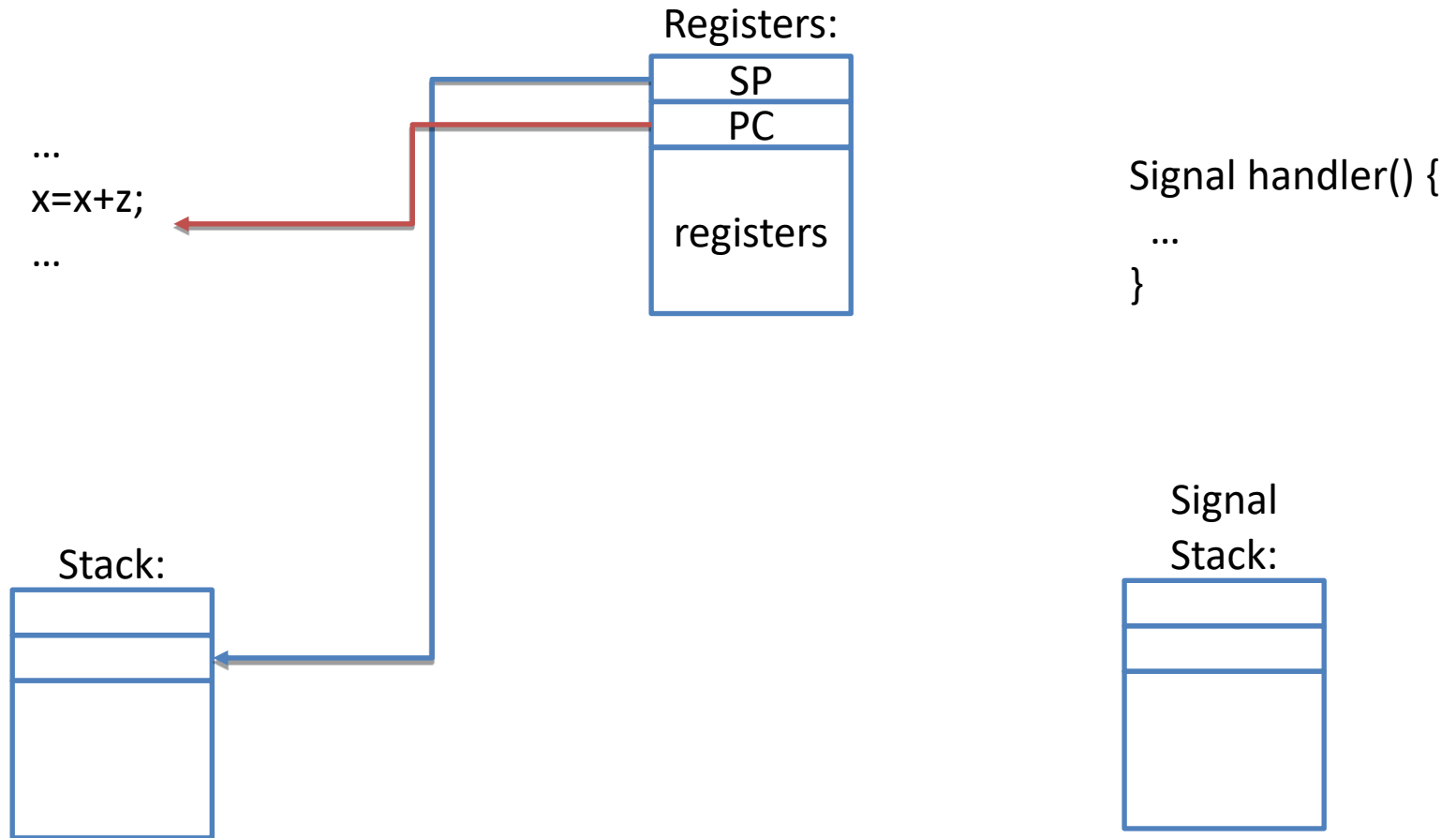
Upcall: During



Upcall: final phase

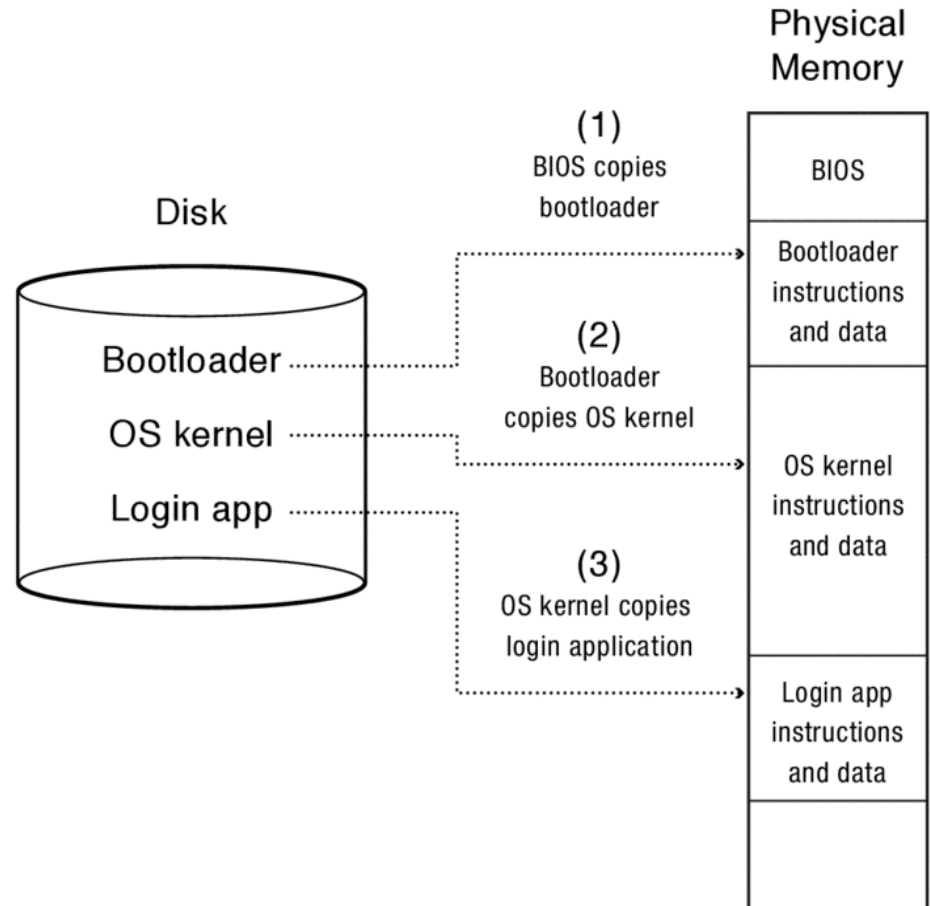


Upcall: end of handler



Kernel Booting

- At boot, the processor executes code in ROM to load the ***first-stage bootloader***.
- The first-stage bootloader initializes the memory controller and a few I/O devices to load in memory a ***second-stage bootloader***.
- The second-stage boot loader loads the kernel and the root file system and then passes control to the kernel



Summary: HW support for OSs

- **Privilege levels**, at least two: user and kernel level.
 - **Privileged instructions**: instructions available only in kernel mode.
 - **Memory translation** prevents user programs from accessing kernel data structures and aids in memory management.
 - **Processor exceptions** trap to the kernel on a privilege violation or other unexpected event.
 - **Timer interrupts** return control to the kernel on time expiration.
 - **Device interrupts** return control to the kernel to signal I/O completion.
 - **Interprocessor interrupts** cause another processor to return control to the kernel.
 - **Interrupt masking** prevents interrupts from being delivered at inopportune times.
 - **System calls** trap to the kernel to perform a privileged action on behalf of a user program.
 - **Return from interrupt**: switch from kernel mode to user mode, to a specific location in a user process.
 - **Boot ROM**: code that loads startup routines from disk into memory.
 - **Support for Virtualization**: hypervisor (aka VMM) and additional privilege levels
- To support threads, we will need one additional mechanism (described later):
- **Atomic read-modify-write instructions** used to implement synchronization in multi-threaded programs.